Identifying Pokémon Trading Cards in Photos: Classical Keypoints, CNN and CLIP Embeddings, and a Hybrid Re-ranking Pipeline

Final Report

Mariana Lozano

CAP 5410 Computer Vision — Final Project (individual)

Department of Computer Science
Florida Polytechnic University, Florida, USA

November 2026

ABSTRACT

Collectors of trading cards need to identify the exact card in a photo: the same Pokémon appears in many versions, sets reprint identical artwork, and phone photos add perspective, glare and blur. This report describes a pipeline built for this task, its systematic evaluation, and an application that puts it into practice. A classical detector finds the card outline (Canny edges, contours, convex hull and quality checks) and warps the card to a canonical 500×700 view with a homography. The rectified card is then matched against a reference database of 785 official scans from 4 recent sets by interchangeable retrieval methods: a pixel-thumbnail baseline, SIFT and ORB keypoints verified with RANSAC, frozen ImageNet ResNet-50 embeddings, frozen CLIP ViT-B/32 embeddings, and a hybrid that re-ranks an embedding shortlist with SIFT and RANSAC. Reprints are handled with artwork groups found by perceptual hashing (761 distinct artworks), and every result is reported both strictly (exact card) and leniently (same artwork), with 95% bootstrap intervals.

On synthetic single-card photos with known ground truth and five distortion levels, exhaustive keypoint matching was the most accurate (strict top-1 at the medium level: ORB 0.970, SIFT 0.930), and its errors at mild distortion were only reprints with identical artwork. Re-ranking a 20-card CLIP shortlist with SIFT raised CLIP from 0.845 to 0.935, matching exhaustive SIFT at about 368 ms instead of 8,941 ms per photo, whereas ResNet-50 alone reached only 0.375. Without rectification the global methods failed almost completely and SIFT dropped to 0.425; under strong distortion or with overlapping cards most errors came from not finding the card outline. In a separate test with English, Spanish and Japanese cards photographed at phone resolution, SIFT with RANSAC was nearly language-independent (lenient top-1 between 0.977 and 1.000), while CLIP alone dropped to 0.767 on Japanese cards; re-ranking the CLIP shortlist with SIFT gave 1.000 with a median of about 553 ms per photo, compared with 33,688 ms for exhaustive SIFT. The pipeline runs in a local web application, which also recognises the 132 cards of a Japanese set and was used to catalogue 7 real cards. On their saved phone photos, SIFT, ORB, CLIP and the CLIP hybrid identified every card, ResNet-50 5 and the pixel baseline only 3, which suggests that the synthetic benchmark favours pixel-level similarity. Two extensions are also evaluated: printed-language recognition with OCR (accuracy 0.911 on the test split) and a visual condition estimate, which works on synthetically damaged cards but not yet on real photos of sleeved cards.

ACKNOWLEDGMENTS

This project received no external funding. It relies on the following external data, code and models, which are gratefully acknowledged.

Data. Card lists, metadata and official card images were obtained from the Pokémon TCG API (https://pokemontcg.io). Missing illustrator names, TCGdex card identifiers, Spanish and Japanese card scans and market prices for the demonstration application were obtained from TCGdex (https://tcgdex.net). Pokémon, the card names and the card artwork are trademarks and copyrights of their respective owners (Nintendo, Creatures Inc. and GAME FREAK inc.); the card images are used here only for non-commercial academic research. **[TODO:** Check the terms of use of both APIs before any public release of the data or the application.**]**

Code libraries. OpenCV (SIFT, ORB, Canny, contours, homography estimation with RANSAC, image warping), NumPy, pandas, PyTorch (Paszke et al., 2019) and torchvision, Hugging Face Transformers (Wolf et al., 2020), EasyOCR (https://github.com/JaidedAI/EasyOCR), Flask and matplotlib.

Pretrained models. ResNet-50 (He et al., 2016) with the ImageNet weights distributed by torchvision (`IMAGENET1K_V2`); CLIP ViT-B/32 (Radford et al., 2021) with the OpenAI weights distributed through Hugging Face (`openai/clip-vit-base-patch32`); the EasyOCR text detector (based on CRAFT (Baek et al., 2019)) and its Japanese, English and Spanish recognition models. None of these models was retrained or fine-tuned in this project.

[TODO: If the course policy requires it, state here (or in the contribution statement) which AI assistants were used, and for what (for example code assistance or drafting of this report).**]**

CONTENTS

ABSTRACT ii

ACKNOWLEDGMENTS iii

CONTENTS iv

LIST OF FIGURES vii

LIST OF TABLES viii

1 INTRODUCTION 1

1.1 PROBLEM STATEMENT 1

1.2 MOTIVATION 2

1.3 CONTRIBUTIONS 2

2 RELATED WORK 3

2.1 LOCAL FEATURES AND GEOMETRIC VERIFICATION 3

2.2 LARGE-SCALE INSTANCE RETRIEVAL 3

2.3 DEEP GLOBAL DESCRIPTORS 4

2.4 DETECTION, RECTIFICATION AND TEXT 4

2.5 CARD RECOGNITION 4

3 TECHNICAL APPROACH 5

3.1 PIPELINE OVERVIEW 5

3.2 CARD LOCALISATION 5

3.3 RECTIFICATION 6

3.4 RETRIEVAL INTERFACE 7

3.5 HYBRID RE-RANKING 7

3.6 VARIANTS 7

3.7 EXTENSIONS USED BY THE APPLICATION 8

4 DATASET AND EXPERIMENTAL SETUP 11
4.1 REFERENCE DATABASE 11
4.2 REPRINTS AND NEAR-DUPLICATES 11
4.3 SYNTHETIC TEST QUERIES 12
4.4 CROSS-LANGUAGE TEST SET 13
4.5 CONDITION VALIDATION SET 13
4.6 METRICS 13
4.7 REAL PHOTOS 14

5 BASELINE AND COMPARISON METHODS 15
5.1 WHY THESE METHODS 16
5.2 EXPERIMENTS 16

6 RESULTS 18
6.1 LOCALISATION AND RECTIFICATION 18
6.2 SINGLE-CARD IDENTIFICATION (MAIN COMPARISON) 19
6.3 SHORTLIST SIZE OF THE HYBRID 20
6.4 EFFECT OF RECTIFICATION 21
6.5 MULTI-CARD SCENES 22
6.6 IDENTIFICATION ACROSS PRINTED LANGUAGES 23
6.7 PRINTED-LANGUAGE CLASSIFICATION 24
6.8 CONDITION ESTIMATE 24
6.9 THE APPLICATION ON REAL PHOTOS 25
6.10 REAL PHOTOS (SYSTEMATIC EVALUATION) 29

7 ANALYSIS AND DISCUSSION 30
7.1 FAILURE ANALYSIS 30
7.2 HOW REALISTIC ARE THE SYNTHETIC RESULTS? 33
7.3 LANGUAGE, TEXT AND THE GLOBAL EMBEDDINGS 34
7.4 WHAT THE APPLICATION SHOWS ABOUT REAL PHOTOS 34
7.5 PRINTED LANGUAGE AND CONDITION 36

8 CONCLUSIONS, LIMITATIONS AND FUTURE WORK 37
8.1 CONCLUSIONS 37
8.2 LIMITATIONS 38
8.3 FUTURE WORK 38

9 CONTRIBUTION STATEMENT 40

REFERENCES 41

APPENDIX A REPRODUCIBILITY AND DETAILED RESULTS 44

LIST OF FIGURES

Figure 3.1 Pipeline overview 5
Figure 3.2 The CardDex application 10

Figure 6.1 Pipeline example 18
Figure 6.2 Accuracy versus distortion severity 19
Figure 6.3 Hybrid shortlist size k (5, 10, 20, 50) at severity 3: accuracy versus median latency, compared with each component alone. 21
Figure 6.4 Language confusion matrices 24
Figure 6.5 Condition estimate example 26
Figure 6.6 Photos saved in the application 27

Figure 7.1 Failure causes per method 31
Figure 7.2 Failure examples 32

LIST OF TABLES

Table 4.1 Reference database 11
Table 4.2 Synthetic distortion severities 13

Table 5.1 Compared methods 15

Table 6.1 Top-1 accuracy per method and severity 19
Table 6.2 Rectification ablation 22
Table 6.3 Multi-card scenes 22
Table 6.4 Identification per printed language 23
Table 6.5 Printed-language classification 24
Table 6.6 Condition estimate validation 25
Table 6.7 Cards saved in the application 26
Table 6.8 Re-identification of the saved photos 28

Table 7.1 Failure causes 30

Table A.1 All finished single-card runs 46

1 INTRODUCTION

Trading card collectors keep track of thousands of cards, and the value of a card depends on its exact version: the set it was printed in, its number inside the set, its rarity, its printed language and its physical condition. Typing this information by hand is slow and error-prone. The goal of this project is to identify the exact card from an ordinary photo taken with a phone, and to build a small application around that capability.

1.1 PROBLEM STATEMENT

Given a photo that contains one (or several) Pokémon trading cards, the system must return, for each card, the identifier of the matching card in a reference database of official card scans (for example `sv9-116`: set *Journey Together*, number 116). Formally, this is *instance-level image retrieval*: the query is the photographed card and the database contains exactly one reference image per card id. The output is a full ranking of the database, so that both top-1 and top-k accuracy can be measured.

Three properties make the problem harder than it first appears:

1. **Near-duplicates and reprints.** A new set can reprint a card of an older set with identical artwork, and many Pokémon appear in several versions (regular, illustration rare, special illustration rare). In the reference database used here, 24 pairs of cards share identical artwork and 163 card names occur more than once (391 cards in total; *Black Belt's Training* alone has 8 versions). A method that recognises only "which Pokémon" is therefore not enough.

2. **Photo conditions.** Phone photos show the card in perspective, rotated, possibly upside down, with uneven lighting, glare on glossy or holographic surfaces, motion blur and sensor noise.

3. **Text and language.** The same card is printed in several languages. The artwork is identical, but the text (name, attacks, rules) differs, so a method that relies on the text area can fail on non-English cards.

1.2 MOTIVATION

Instance retrieval of flat, textured objects is a classical computer-vision problem, and trading cards are a good test bed to compare the main families of methods on the same data: hand-crafted local features with geometric verification (Fischler and Bolles, 1981; Lowe, 2004; Rublee et al., 2011), global embeddings from a convolutional network trained on ImageNet (He et al., 2016), and embeddings from a vision transformer trained with language supervision (Dosovitskiy et al., 2021; Radford et al., 2021). The comparison is interesting because the methods have opposite strengths: local features are precise but slow when matched exhaustively, while global embeddings are fast but may confuse visually similar cards. A hybrid that uses the embedding to build a short candidate list and local features to verify it is a natural compromise, and the project measures how much of each strength it keeps.

1.3 CONTRIBUTIONS

The contributions of this project are:

- a reference database of 785 cards from 4 recent sets, with reprints grouped by artwork using perceptual hashing, and two evaluation metrics (strict and lenient) that handle reprints explicitly (Chapter 4);
- a classical card detection and rectification stage, and a common retrieval interface implemented for seven methods and their variants (Chapters 3 and 5);
- a synthetic query generator with five distortion severities and exact ground truth, and an evaluation that logs every query with 95% bootstrap confidence intervals;
- a systematic comparison of all methods across the five severities, a study of the hybrid's shortlist size, a rectification ablation, multi-card scenes, a cross-language test and an automatic failure analysis (Chapters 6 and 7);
- two extensions with their own validation: printed-language recognition with OCR and a visual condition estimate;
- a working local web application (CardDex) that integrates all of the above, recognises English and Japanese printings, and was used to identify and catalogue real cards; the photos saved in it serve as a small real-photo test of all methods (Section 6.9).

2 RELATED WORK

2.1 LOCAL FEATURES AND GEOMETRIC VERIFICATION

The scale-invariant feature transform (SIFT) of Lowe (2004) detects keypoints as extrema of a difference-of-Gaussians scale space and describes each one with a 128-dimensional histogram of gradient orientations. Lowe also introduced the *ratio test*, which keeps a match only if the nearest descriptor is clearly closer than the second nearest, and showed that recognising an object from a few verified matches is reliable even under occlusion and clutter. ORB (Rublee et al., 2011) replaces SIFT with an oriented FAST detector and a rotated binary BRIEF descriptor compared with the Hamming distance, which makes it much faster to compute and match. Random sample consensus (RANSAC) (Fischler and Bolles, 1981) fits a model, here a planar homography, to data with many outliers by repeatedly fitting minimal samples and keeping the model with the largest set of inliers. For flat objects such as cards, the number of RANSAC inliers of a homography is a strong similarity score, because random matches almost never agree on one plane-to-plane mapping.

2.2 LARGE-SCALE INSTANCE RETRIEVAL

Matching a query against every database image is expensive. Sivic and Zisserman (2003) quantised local descriptors into "visual words" and borrowed inverted files and tf-idf weighting from text retrieval; Nistér and Stewénius (2006) scaled this with a hierarchical vocabulary tree and demonstrated it on CD covers, a problem very close to cards. Philbin et al. (2007) added *spatial re-ranking*: a fast first stage produces a shortlist, and only the shortlist is verified geometrically. The hybrid method of this project follows the same two-stage idea, with a global embedding instead of a bag of visual words as the first stage. Benchmarks such as the revisited Oxford and Paris datasets (Radenović et al., 2018) remain the standard way to compare instance retrieval methods, and they report mean average precision, which with a single relevant image per query equals the mean reciprocal rank used here.

2.3 DEEP GLOBAL DESCRIPTORS

Activations of convolutional networks trained for ImageNet classification (Deng et al., 2009) turned out to be strong off-the-shelf descriptors for retrieval (Babenko et al., 2014; Razavian et al., 2014). Later work fine-tuned such networks specifically for retrieval with ranking losses (Gordo et al., 2016; Radenović et al., 2019), and DELG (Cao et al., 2020) unified a global descriptor for the first stage with local descriptors for geometric re-ranking in a single network. This project uses *frozen* pretrained models without fine-tuning: a residual network (He et al., 2016) and the image encoder of CLIP (Radford et al., 2021), a vision transformer (Dosovitskiy et al., 2021) trained contrastively on image–text pairs. Using frozen models keeps the comparison simple and avoids the need for labelled photos of every card, but it means that the embeddings were never trained to separate near-identical card versions.

2.4 DETECTION, RECTIFICATION AND TEXT

Before retrieval, the card must be found and seen from the front. The detector used here is classical: Canny edge detection (Canny, 1986), border following to extract contours (Suzuki and Abe, 1985), and polygon simplification (Douglas and Peucker, 1973) to obtain four corners, followed by a perspective warp. Printed-language recognition uses the EasyOCR toolkit, whose text detector is CRAFT (Baek et al., 2019). Reprints are grouped with a perceptual hash of the image based on the discrete cosine transform, in the spirit of robust image hashing (Venkatesan et al., 2000). The statistical tools are the bootstrap (Efron, 1979) for confidence intervals and Spearman's rank correlation (Spearman, 1904) for the condition validation.

2.5 CARD RECOGNITION

Commercial apps for trading card recognition exist, but their methods are not published. Chandrasekhar et al. (2011) released a mobile visual search dataset of phone photos of flat products (such as CD covers, book covers and business cards) matched against clean reference images, which is the most similar published setting found so far. **[TODO:** Search IEEE Xplore, ACM Digital Library and Google Scholar for peer-reviewed work specifically on trading-card (Pokémon, Magic: The Gathering, sports cards) recognition, and add it here. Only peer-reviewed sources may be cited.**]**

3 TECHNICAL APPROACH

3.1 PIPELINE OVERVIEW

Figure 3.1 shows the complete pipeline. A photo first goes through *localisation*, which finds the outline of each card as a quadrilateral, and *rectification*, which warps each card to a canonical front view of 500×700 pixels (the 5:7 shape of a 63 mm $\times$ 88 mm card). The rectified card is then passed to a *retrieval method*, which ranks all reference cards. All retrieval methods share one interface, so that every method receives exactly the same rectified input and is evaluated by the same code. Language recognition and the condition estimate run after identification, in the application.

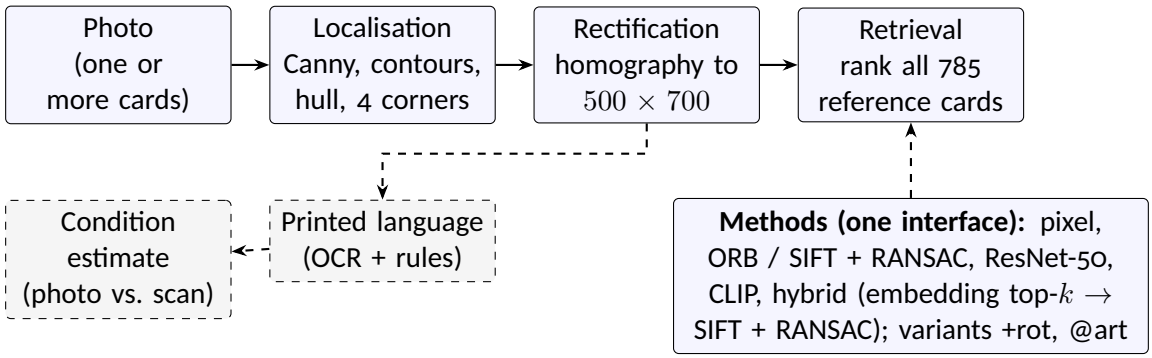


Figure 3.1. Pipeline overview. Solid arrows: the identification pipeline that is evaluated in this report. Dashed boxes: extensions used by the application (Section 3.7).

3.2 CARD LOCALISATION

The detector (`cardrec/rectify.py`) is fully classical and has no learned parameters. Its design goal is to find the *outer* outline of the card, not an inner frame such as the artwork box, under perspective, blur and glare.

1. **Edge maps.** The photo is downscaled so that its longest side is at most 800 pixels and

smoothed with a 5×5 Gaussian filter. Two channels are used: grey level and the HSV saturation, because a silver card border can have the same brightness as the table but a different colourfulness. On each channel the Canny detector (Canny, 1986) is run with three threshold pairs (fixed $50/150$, relative to the median grey level $0.5m/1.0m$, and low $10/40$ for blurry photos), and each edge map is dilated with a 3×3 kernel once (keeps the thin gap between touching cards) and twice (closes gaps in blurry edges). All twelve edge maps are used.

2. **Candidate quadrilaterals.** Contours are extracted from every edge map (Suzuki and Abe, 1985). For each contour, its convex hull is simplified with the Douglas–Peucker algorithm (Douglas and Peucker, 1973), with a tolerance that grows from 2% to 8% of the perimeter until four corners remain. Using the hull instead of the raw contour makes the step robust to rounded card corners and to edges broken by glare.

3. **Checks.** A quadrilateral is kept only if (i) its area is at least 3% of the image, (ii) it does not touch the image border, (iii) its aspect ratio (long side / short side) is between 1.15 and 1.95 (a real card is $88/63 \approx 1.40$; perspective changes it), (iv) the contour fills at least 85% of the quadrilateral, and (v) its four sides lie on real edges: of 60 points sampled along each side, at least 60% must be edge pixels ("edge support"). The last check rejects large shapes stretched around background noise.

4. **Selection.** Candidates are sorted by score $=$ area $\times$ support^2, so that a quadrilateral that also covers some background (lower support) loses to the true card outline. A candidate is dropped if its centre lies inside an already accepted card (inner frames) or if it overlaps one with intersection-over-union (IoU) above 0.5 (duplicates from different edge maps). Up to ten cards are returned.

If no candidate passes, the single-card pipeline falls back to resizing the whole photo to 500×700. The thresholds were tuned only on separate development query sets, never on the test queries described in Chapter 4.

3.3 RECTIFICATION

The four corners are ordered by their angle around the centroid and the list starts at the corner with the smallest $x + y$, which becomes the top-left corner of the output. If the top side is longer than the left side, the card is lying sideways and the order is rotated by one position. The perspective transform H that maps the four corners $\mathbf{p}_i$ to the corners of the 500×700 rectangle $\mathbf{q}_i$ is computed exactly from the four correspondences,

$$\lambda_i \begin{pmatrix} \mathbf{q}_i \\ 1 \end{pmatrix} = H \begin{pmatrix} \mathbf{p}_i \\ 1 \end{pmatrix}, \qquad i = 1, \ldots, 4, \tag{3.1}$$

and the photo is warped with H. This step cannot tell the top of the card from the bottom: a card rotated by 180° produces a valid but upside-down rectification. With the corner ordering above, cards rotated by about -45° to $+135^\circ$ come out upright and other rotations come out upside

down. Keypoint methods are rotation invariant and do not care; for the embedding methods, the "+rot" variant (Section 3.6) addresses it.

3.4 RETRIEVAL INTERFACE

Every method implements two operations: `build_index(reference images, ids)`, run once, and `query(rectified card)`, which returns the *complete* ranking of the database as (id, score) pairs. Reference scans are resized to the same 500×700 canonical size as the queries. Indexes are cached on disk, so the expensive index of the keypoint methods is computed only once.

Keypoint matching with RANSAC. For SIFT (Lowe, 2004) and ORB (Rublee et al., 2011), up to 800 keypoints are detected on the grey-level card. Each query descriptor is matched to its two nearest neighbours in a reference card by brute force (L2 distance for SIFT, Hamming distance for ORB), and a match is kept if it passes the ratio test with ratio 0.75. If at least four matches remain, a homography is estimated with RANSAC (Fischler and Bolles, 1981) using a reprojection threshold of 5 pixels, and the *number of inliers* is the similarity score. The query is compared with every reference card.

Global embeddings. An embedding method maps an image to a vector $\mathbf{f}$, normalised to unit length. The similarity between the query $\mathbf{f}_q$ and a reference $\mathbf{f}_r$ is the cosine similarity $s(q, r) = \mathbf{f}_q^\top \mathbf{f}_r$, computed for all references with one matrix product.

3.5 HYBRID RE-RANKING

The hybrid combines a fast embedding with precise geometric verification, in the spirit of spatial re-ranking (Philbin et al., 2007). Let S_k be the k references with the highest embedding similarity to the query. Only these k cards are verified with SIFT + RANSAC, and they are re-ordered by

$$\text{rank within } S_k \text{ by } \big(\, \text{inliers}(q, r)\downarrow,\ \ s(q, r)\downarrow \,\big), \tag{3.2}$$

that is, by the number of RANSAC inliers, with ties broken by the embedding similarity. The cards outside the shortlist keep their embedding order after the k re-ranked cards, so the output is still a full ranking. The default shortlist size is $k = 20$, so SIFT matching is done against 20 cards instead of 785. If the correct card is not in the shortlist, the hybrid cannot recover it; this is why the shortlist size is a parameter of the experiments.

3.6 VARIANTS

- **+rot** (embedding and hybrid methods only). The query is embedded twice, as rectified and rotated by $180°$, and each reference keeps the higher of the two similarities. This removes the up/down ambiguity of the rectification at the cost of a second forward pass.

- **@art**. Both the references and the query are cropped to a fixed artwork window (rows 70–340, columns 35–465 of the 500×700 card) before indexing and querying. The window contains no text, so it is the same in every language. On full-art cards it is simply the upper-middle part of the illustration.

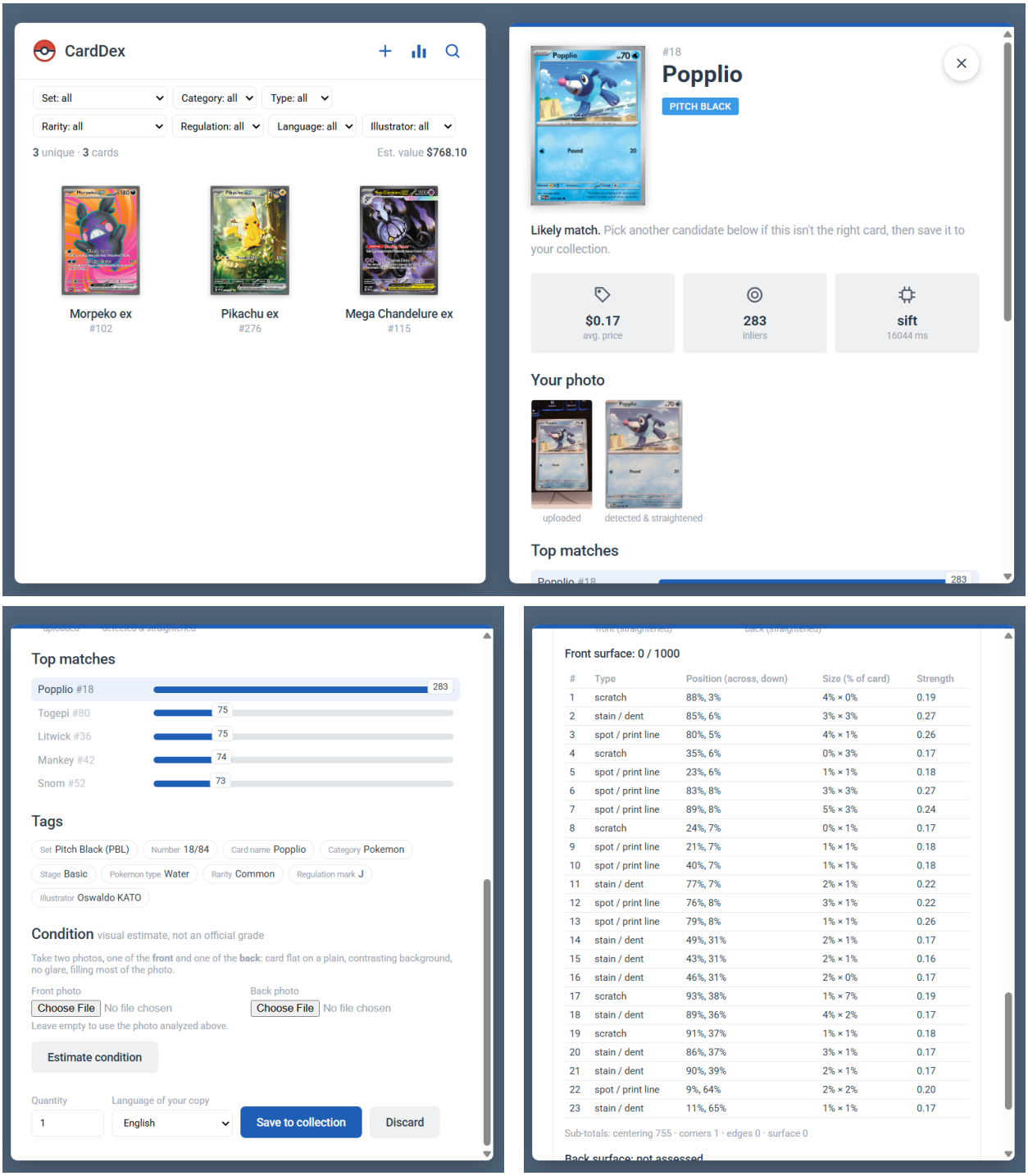
3.7 EXTENSIONS USED BY THE APPLICATION

Printed language. Identification and language are treated as separate problems. The card is identified from its image (for non-English cards, preferably with an @art method), and the printed language is classified separately (`cardrec/language.py`). The rectified card is re-warped at 750×1050 pixels, because the small text is unreadable at 500×700, and read by EasyOCR twice, once with a Japanese+English reader and once with a Spanish+English reader. Fixed rules then decide: (1) if kana characters (hiragana and katakana) make up at least 30% of the letters read by the Japanese reader, the card is Japanese (kanji are not counted, because the reader invents kanji on blurry Latin text); (2) otherwise, the language code printed next to the set code at the bottom left (for example "PRE EN") is read in an enlarged crop; (3) otherwise, typical English and Spanish words and Spanish accented characters are counted, and the larger count wins; with no evidence the answer is "unknown".

Visual condition estimate. Once a card is identified, its official scan is a picture of the undamaged card. The condition module (`cardrec/condition.py`) re-detects the outline precisely, warps the photo to 750×1050 pixels (about 12 pixels per millimetre), aligns the scan to the photo with SIFT + RANSAC and matches the colours. It then measures centering from the border widths, three sub-scores (whitening, fill, fray) for each of the four corners and four edges, and surface defects as remaining differences between photo and scan. These are combined into a 0–1000 score with hand-set weights. The result is explicitly labelled as an *estimate*, not an official grade.

Application. A local web application called CardDex (Flask back end, single-page front end, SQLite collection) puts the pipeline into practice (Figure 3.2). The user uploads a photo and chooses one of eight retrieval methods (SIFT by default); a one-sentence description of each method, with its typical speed, is shown next to the selector, so that a user without a computer-vision background can choose, for example, the fast CLIP hybrid for everyday use or the artwork-only hybrid for non-English cards. The application detects and rectifies the card, ranks the reference cards with the selected method and shows the top-5 candidates with their scores, so that a wrong first guess can be corrected by clicking another candidate. For inlier-based methods it marks a result as a likely match when the best card has at least 15 inliers and at least 1.4 times the inliers of the second card. It then shows tags derived from the card metadata (set, number, name, category, stage, type, rarity, regulation mark, illustrator), classifies the printed language, optionally estimates the condition from a front and a back photo, and saves the card to a personal collection with its quantity, language and a market price (TCGdex first, Pokémon TCG API as fallback, cached for 24 hours). The collection panel can be filtered by every tag. Besides the 785 English reference cards, the application's database contains the 132 cards of the Japanese set *Battle Partners* (official

scans from TCGdex), so that Japanese copies can be identified as Japanese cards and not only as their English counterparts. 128 of them were linked to the English card with the same artwork using the rule of the cross-language test set (Section 4.4), and all links were checked by eye. When the best match is a Japanese card, the application pre-selects Japanese as the language of the user's copy, and its price is taken from TCGdex's Japanese catalogue (Cardmarket, in EUR). These Japanese cards are used only by the application, not in the benchmark experiments of this report. The application is started with `run_app.bat` or `python -m cardrec.app.server`; it runs locally and needs internet access only for prices.



(a) Top-5 matches, tags and saving.

(b) Condition report: surface table.

Figure 3.2. The CardDex application (screenshots by the author on 1 November 2026, browser frame removed; the collection shown is the one at that date). Top: collection with tag filters (left) and the analysis of an uploaded phone photo (right): best match, average price, number of RANSAC inliers, method and time, and the uploaded and straightened photo. (a) The ranked candidates, the tags taken from the database and the save form. (b) Part of the condition report of the same card.

4 DATASET AND EXPERIMENTAL SETUP

4.1 REFERENCE DATABASE

The reference database contains one official scan per card for 4 recent sets of the Pokémon Trading Card Game (Table 4.1): 785 unique card ids in total, of which 629 are Pokémon cards, 150 Trainer cards and 6 Energy cards. The counts include the secret rares, which are numbered above the printed set size. Card lists, metadata and images come from the Pokémon TCG API; illustrator names missing in that source and the Spanish and Japanese scans used in the language test come from TCGdex. The download script (`cardrec/database.py`) writes a manifest with one row per card (id, name, set, number, supertype, subtypes, types, rarity, regulation mark, illustrator, image path, artwork group).

Table 4.1. Reference database: one official scan per card id. "Printed" is the set size printed on the cards; the extra cards are secret rares numbered above it.

Set (API id)	Code	Cards	Printed
Prismatic Evolutions (`sv8pt5`)	PRE	180	131
Journey Together (`sv9`)	JTG	190	159
Ascended Heroes (`me2pt5`)	ASC	295	217
Pitch Black (`me5`)	PBL	120	84
Total		785	591

4.2 REPRINTS AND NEAR-DUPLICATES

Two kinds of near-duplicates are handled differently.

Identical artwork (reprints). Some cards are reprinted in a later set with exactly the same artwork; only the small set symbol, set code and collector number differ. Such pairs are grouped automatically (`cardrec/art_groups.py`) with a 64-bit perceptual hash: the grey card is resized

to 32×32, transformed with the discrete cosine transform, and the 8×8 lowest-frequency coefficients are thresholded at their median, giving one bit each. Two cards are considered the same artwork if *both* the hash of the whole card and the hash of the artwork window differ in at most 6 bits (Hamming distance). Groups are then formed transitively with union–find. This produced 24 same-artwork pairs, all between *Ascended Heroes* and an older set (9 with *Prismatic Evolutions* and 15 with *Journey Together*), so the 785 cards contain 761 distinct artworks. The largest distances inside accepted pairs were 2 bits (whole card) and 6 bits (artwork window). All pairs were checked by eye on a generated contact sheet.

Lookalikes. Cards with the same name but different artwork (for example the 8 versions of *Black Belt's Training*, or a regular card and its illustration rare) are *not* grouped: they are different cards that a correct system must tell apart. The database has 163 names that occur on more than one card (391 cards).

Metrics that follow from this. Every query is scored twice:

- **strict**: correct only if the predicted id is exactly the true id;
- **lenient**: correct if the predicted card is in the same artwork group as the true card (a reprint counts as correct).

The difference between the two metrics measures exactly how many errors are reprint confusions.

4.3 SYNTHETIC TEST QUERIES

Real labelled photos of all 785 cards do not exist, so the main experiments use synthetic photos with exact ground truth (`cardrec/synth.py`). A reference scan is placed on a 1280×960 background (a random smooth colour field plus noise) with a random position, in-plane rotation and a random perturbation of its four corners (perspective). The whole image then receives a lighting gradient with a gamma change, a glare blob centred on the card, Gaussian or motion blur, and Gaussian noise, and is saved as JPEG (quality 92). The strength of each distortion is set by a severity level from 1 to 5 (Table 4.2); at severity 5 the rotation covers the full circle. The true corner positions are stored with each query, so the accuracy of the detector can be measured as well.

For each severity, 200 different cards are drawn at random without replacement (fixed seed), so each test set covers about a quarter of the database. Note that the cards differ between severities, so a comparison across severities also involves different cards. Separate development query sets with other seeds were used to tune the detector; the test queries were never used for tuning.

Table 4.2. Distortion parameters per severity level (`cardrec/synth.py`). Each value is the maximum of a random draw.

Parameter	S1	S2	S3	S4	S5
Max. in-plane rotation ($\pm$deg)	5	15	30	60	180
Corner jitter (frac. of card height)	0.02	0.04	0.07	0.1	0.14
Blur σ (px, Gaussian or motion)	0	1	1.8	2.8	4
Gaussian noise σ (grey levels)	2	5	8	12	18
Lighting gradient + gamma strength	0.1	0.2	0.3	0.4	0.5
Glare blob strength	0	0.2	0.4	0.6	0.8

4.4 CROSS-LANGUAGE TEST SET

To test printed languages, a separate set was built from official scans (`cardrec/lang_data.py`) with two disjoint splits, a development split for tuning the language rules and a test split for the reported numbers. The test split has 60 English cards from the database, 60 Spanish cards (same card ids as their English versions) and 60 Japanese cards from the two Japanese sets that correspond to *Prismatic Evolutions* and *Journey Together*. Japanese numbering differs, so the matching English card was found *without* the tested methods: same illustrator and same Pokédex number, then the closest artwork hash, accepted only up to a distance of 14 bits. This gave an English ground truth for 43 of the 60 Japanese cards; all matches were checked by eye, and the remaining Japanese cards are used only for language classification. Synthetic photos were rendered at severity 2 at two resolutions: the standard canvas (card about 530 pixels tall, "low") and a 2560×1920 canvas (card about 1050 pixels tall, similar to a 12-megapixel phone photo, "phone").

4.5 CONDITION VALIDATION SET

The condition estimate was tuned on 40 development cards and is validated on 80 test cards drawn with a different random seed (the two samples were not forced to be disjoint: 2 cards appear in both, with different random damage). Each official scan is damaged with known strengths in $\{0, 0.25, 0.5, 0.75, 1\}$: corner whitening, corner cuts, edge whitening, edge fraying, scratches and stains at known positions, and a shift of the print of up to about 1 mm (centering). The damaged card is rendered as a phone-resolution photo at severity 1 and analysed exactly like a real photo. Only the front side can be validated, because no official scans of the card back exist.

4.6 METRICS

- **Top-1 / top-5**: fraction of queries whose correct card is ranked first / among the first five.
- **MRR**: mean reciprocal rank $\frac{1}{N}\sum_i 1/\text{rank}_i$. With exactly one relevant card per query, mean average precision equals the MRR.

- **95% confidence intervals**: percentile bootstrap (Efron, 1979) over the queries, 1000 resamples.
- **Outline accuracy**: IoU between the detected and the true card polygons; a rectification counts as correct if IoU ≥ 0.8, and as a localisation failure if IoU < 0.5.
- **Latency**: median time per photo for rectification plus retrieval. The experiments ran on a laptop that was running other jobs (including other experiments) at the same time, so the latencies indicate orders of magnitude, not exact times; accuracy is not affected.

Every query of every run is logged (predicted id, rank of the true card, outline IoU, timings), and all tables and figures of this report are regenerated from these logs by a script (`report/make_figures.py`).

4.7 REAL PHOTOS

A protocol and labelling tools for real phone photos are ready (`cardrec/real_photos.py`): 70–100 photos per set of physically owned cards, with deliberately varied backgrounds, lighting, glare, sleeves, rotation, and 20–30 photos with several cards; the ground truth must be entered by hand and not taken from any tested method. This systematic set was not collected within the project. The real photos used in this report are the 7 photos that were saved with the identified cards in the application (Section 6.9); for these, the ground truth is the card that the author confirmed in the application before saving.

5 BASELINE AND COMPARISON METHODS

All methods receive the same rectified 500×700 card and return a full ranking of the 785 reference cards. Table 5.1 lists them with the settings used in the code. No method is trained or fine-tuned on card data; the deep models are used frozen with their public pretrained weights.

Table 5.1. Compared retrieval methods and their settings as implemented (`cardrec/methods/`).

Method	Representation	Similarity
Pixel thumbnail (sanity baseline)	32×44 grey thumbnail, mean subtracted (1408-d)	cosine
ORB + RANSAC	up to 800 ORB keypoints, binary descriptors	ratio test 0.75, RANSAC homography inliers (5 px), against every card
SIFT + RANSAC	up to 800 SIFT keypoints, 128-d descriptors	as ORB, with L2 distance
ResNet-50	ImageNet ResNet-50, classification layer removed, global average pooled 2048-d vector; input: whole card resized to 224×320	cosine
CLIP ViT-B/32	CLIP image encoder, 512-d image embedding; default preprocessing (short side to 224, centre crop 224×224)	cosine
Hybrid ResNet$\rightarrow$SIFT, Hybrid CLIP$\rightarrow$SIFT	top-k shortlist from the embedding ($k = 20$ by default)	SIFT + RANSAC inliers inside the shortlist, ties by cosine
+rot (embedding methods)	also embed the query rotated 180°	best of the two
@art (any method)	only the artwork window of query and references	as the base method

5.1 WHY THESE METHODS

Pixel thumbnail. A deliberately weak baseline. If the pipeline only had to undo a perfect warp of the reference image, comparing tiny thumbnails would already work; its accuracy therefore shows how easy the synthetic queries are once the card is rectified, and it gives a lower bound for the other methods.

ORB and SIFT with RANSAC. The classical approach to instance recognition of flat objects (Fischler and Bolles, 1981; Lowe, 2004; Rublee et al., 2011). They are rotation invariant, use fine details (the collector number, the set symbol) as well as the artwork, and verify geometric consistency, which makes them precise. Their cost grows linearly with the database because every reference card is matched; ORB is the cheaper binary variant.

ResNet-50. A convolutional network pretrained on ImageNet (Deng et al., 2009; He et al., 2016), used as an off-the-shelf global descriptor (Babenko et al., 2014; Razavian et al., 2014). It sees the whole card with its 5:7 shape.

CLIP ViT-B/32. The image encoder of CLIP (Radford et al., 2021), a vision transformer (Dosovitskiy et al., 2021) trained on image–text pairs. Its default preprocessing crops the centre square of the card, so the top and bottom of the card (about one seventh each, including the name bar and the collector number) are not seen. This is the preprocessing of the released model and was kept on purpose, but it is a limitation for telling card versions apart.

Hybrids. The embedding chooses k candidates, and SIFT + RANSAC decides among them (Section 3.5). The expected benefit is SIFT-level precision at a fraction of its cost, as long as the correct card is in the shortlist.

5.2 EXPERIMENTS

The experiment script (`cardrec/experiments.py`) defines four comparisons:

1. **main**: the eleven configurations of Table 6.1 (pixel, ORB, SIFT, ResNet-50, CLIP, both hybrids, and the +rot variants of the last four) at all five severities, 200 queries each;
2. **k-sweep**: both hybrids with $k \in \{5, 10, 20, 50\}$ at severity 3, accuracy versus latency;
3. **no-rectify ablation**: every configuration at severity 3 with the whole photo resized instead of rectified;
4. **scenes**: multi-card scenes with 2–6 cards and maximum overlaps of 0%, 15% and 30%, evaluated with ResNet-50, CLIP and both hybrids.

All four are produced by `python -m cardrec.experiments --exp all`; each run logs every query, so all tables and figures of Chapter 6 are regenerated from the logs by `report/make_figures.py`.

6 RESULTS

This chapter reports the results of all experiments. All accuracies are on the synthetic test queries unless stated otherwise; real photos are covered in Section 6.9.

6.1 LOCALISATION AND RECTIFICATION

The detector is the same for every retrieval method, so its accuracy on the test queries is a property of the query set. Figure 6.1 shows a typical successful case. On the 200 single-card test queries per severity, the detected outline was correct (IoU ≥ 0.8) for 98.5%, 99.0%, 98.0%, 89.0% and 27.0% of the queries at severities 1 to 5. At severities 4 and 5 the detector returned a card outline at all for only 91.0% and 36.0% of the queries; in the remaining cases the pipeline fell back to the whole photo.

(a) synthetic query, severity 3 (b) rectified 500×700 (c) top-1: sv8pt5-96

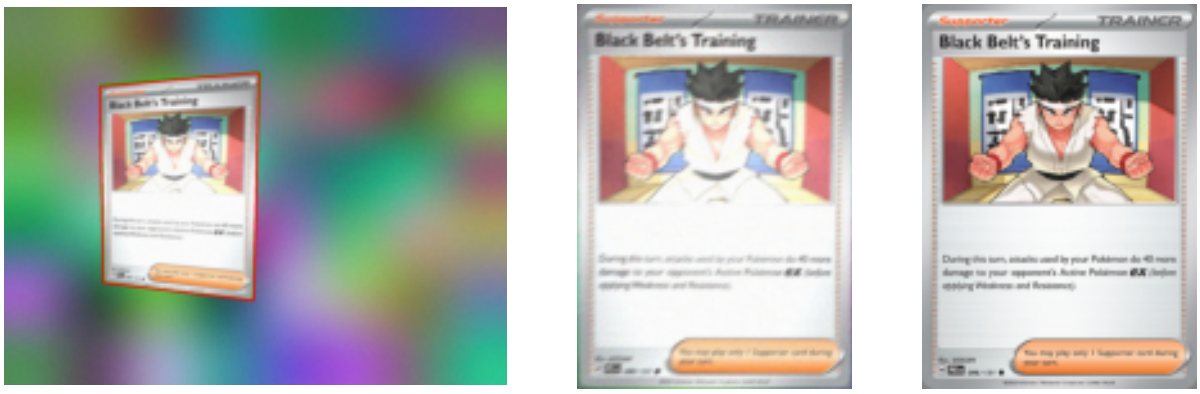


Figure 6.1. Example run of the pipeline. (a) Synthetic query `s3_0000`; green = true outline, red = outline found by the detector. (b) The card warped to the canonical view. (c) The reference scan ranked first by ORB + RANSAC (correct; 145 RANSAC inliers versus 67 for the second card).

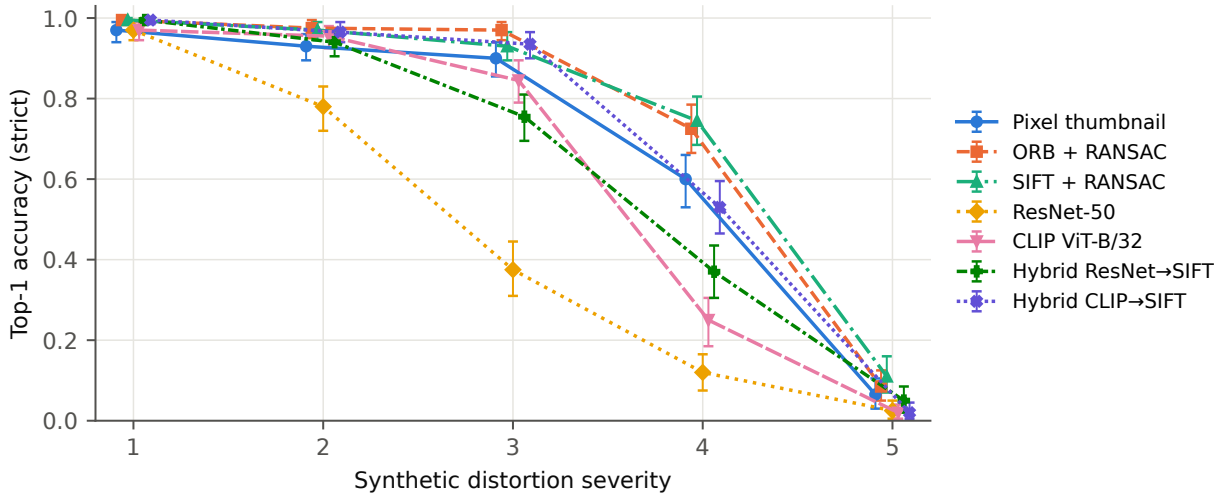


Figure 6.2. Strict top-1 accuracy versus synthetic distortion severity for the seven base methods, with 95% bootstrap intervals (200 queries per point; points slightly shifted horizontally for readability; the +rot variants are in Table 6.1).

6.2 SINGLE-CARD IDENTIFICATION (MAIN COMPARISON)

Table 6.1 and Figure 6.2 show the strict and lenient top-1 accuracy of every method at every severity; Table A.1 in Appendix A adds top-5, MRR, 95% confidence intervals and latencies.

Table 6.1. Top-1 identification accuracy on the synthetic single-card test queries (200 per severity), shown as strict / lenient. Strict = exact card id; lenient = any card with the same artwork (reprint). 95% bootstrap intervals are in Table A.1 and Figure 6.2.

Method	S1	S2	S3	S4	S5
Pixel thumbnail	0.970 / 0.980	0.930 / 0.950	0.900 / 0.930	0.600 / 0.615	0.065 / 0.070
ORB + RANSAC	0.995 / 1.000	0.975 / 1.000	0.970 / 0.995	0.725 / 0.740	0.085 / 0.085
SIFT + RANSAC	0.995 / 1.000	0.970 / 1.000	0.930 / 0.950	0.745 / 0.765	0.110 / 0.125
ResNet-50	0.970 / 0.995	0.780 / 0.800	0.375 / 0.380	0.120 / 0.120	0.025 / 0.025
CLIP ViT-B/32	0.970 / 1.000	0.955 / 0.995	0.845 / 0.875	0.250 / 0.255	0.020 / 0.020
Hybrid ResNet→SIFT	0.995 / 1.000	0.940 / 0.970	0.755 / 0.780	0.370 / 0.385	0.050 / 0.050
Hybrid CLIP→SIFT	0.995 / 1.000	0.965 / 0.995	0.935 / 0.955	0.530 / 0.535	0.020 / 0.020
ResNet-50 +rot	0.970 / 0.995	0.745 / 0.760	0.340 / 0.345	0.130 / 0.130	0.030 / 0.030
CLIP +rot	0.970 / 1.000	0.950 / 0.990	0.810 / 0.840	0.275 / 0.280	0.020 / 0.020
Hybrid ResNet→SIFT +rot	0.995 / 1.000	0.940 / 0.970	0.715 / 0.735	0.370 / 0.375	0.055 / 0.055
Hybrid CLIP→SIFT +rot	0.995 / 1.000	0.965 / 0.995	0.925 / 0.945	0.565 / 0.570	0.040 / 0.040

Mild distortion (severities 1–2). All methods except ResNet-50 were close to perfect. At severity 1, ORB, SIFT and both hybrids reached a strict top-1 of 0.995, 0.995, 0.995 and 0.995, and ResNet-50 and CLIP 0.970 and 0.970. At severity 2 ResNet-50 already dropped to 0.780, while CLIP kept

0.955 and the CLIP hybrid 0.965. Under the lenient metric, ORB reached 1.000 and 1.000 and SIFT 1.000 and 1.000, so every strict error of ORB and SIFT at these severities was a confusion with a reprint of the same artwork.

Moderate distortion (severity 3). The methods separate clearly. ORB + RANSAC reached 0.970 [0.945, 0.990], SIFT + RANSAC 0.930 [0.895, 0.965], the CLIP hybrid 0.935 [0.900, 0.965], CLIP alone 0.845, the ResNet hybrid 0.755 and ResNet-50 alone only 0.375. Re-ranking the shortlist with SIFT therefore raised CLIP from 0.845 to 0.935 and ResNet-50 from 0.375 to 0.755. The pixel-thumbnail baseline reached 0.900, more than both global deep embeddings alone; this is discussed in Section 7.2.

Strong distortion (severities 4–5). At severity 4, the two exhaustive keypoint methods remained best, SIFT with 0.745 [0.685, 0.805] and ORB with 0.725 [0.665, 0.785] (overlapping intervals), followed by the pixel baseline (0.600), the CLIP hybrid with rotation (0.565) and without (0.530), the ResNet hybrid (0.370), CLIP (0.250) and ResNet-50 (0.120). At severity 5 every method failed (strict top-1 between 0.020 for CLIP and 0.110 for SIFT), because the card outline was found correctly in only 27.0% of these photos (Section 7.1).

Trying both orientations (+rot). Embedding the query also rotated by 180° removed every "upside down" failure (Table 7.1), but it did not improve the accuracy: at severity 3 it was slightly lower for every embedding method (for example CLIP 0.810 versus 0.845, CLIP hybrid 0.925 versus 0.935), and at severity 4 slightly higher for CLIP and its hybrid (0.275 versus 0.250, 0.565 versus 0.530); in every case the 95% intervals of the two variants overlap. A plausible explanation is that comparing each reference with two versions of the query also doubles the chances of a wrong card scoring high. In the failure analysis, pooled over all severities, the number of failures with and without +rot was 557 versus 546 for ResNet-50, 395 versus 392 for CLIP, 385 versus 378 for the ResNet hybrid and 302 versus 311 for the CLIP hybrid: the upside-down failures disappear, but other failures take most of their place.

ORB versus SIFT. The two keypoint methods were statistically tied at every severity: ORB was slightly ahead at severity 3 (0.970 versus 0.930) and SIFT slightly ahead at severities 4 and 5 (0.745 versus 0.725, 0.110 versus 0.085), with overlapping 95% intervals in every case. Both match the query against all 785 reference cards, which took several seconds per photo on the test laptop (median 6,981 ms for ORB and 8,941 ms for SIFT at severity 3).

6.3 SHORTLIST SIZE OF THE HYBRID

Figure 6.3 shows how the shortlist size k trades accuracy for time at severity 3. For the CLIP hybrid the shortlist matters little: strict top-1 grew from 0.910 with $k = 5$ (median 234 ms per photo) to 0.935 with $k = 20$ (414 ms) and 0.950 with $k = 50$ (833 ms). Already with $k = 20$ the hybrid matched exhaustive SIFT (0.930, median 8,941 ms per photo), which uses exactly the same SIFT

verification but against all 785 cards instead of 20; with $k = 50$ its 95% interval [0.920, 0.980] also overlaps that of exhaustive ORB (0.970 [0.945, 0.990], median 6,981 ms), in a fraction of the time. For the ResNet hybrid the shortlist is decisive: 0.565 with $k = 5$, 0.755 with $k = 20$ and 0.880 with $k = 50$, because ResNet-50 often does not rank the correct card among its first few candidates, and SIFT can only choose among the cards it is given.

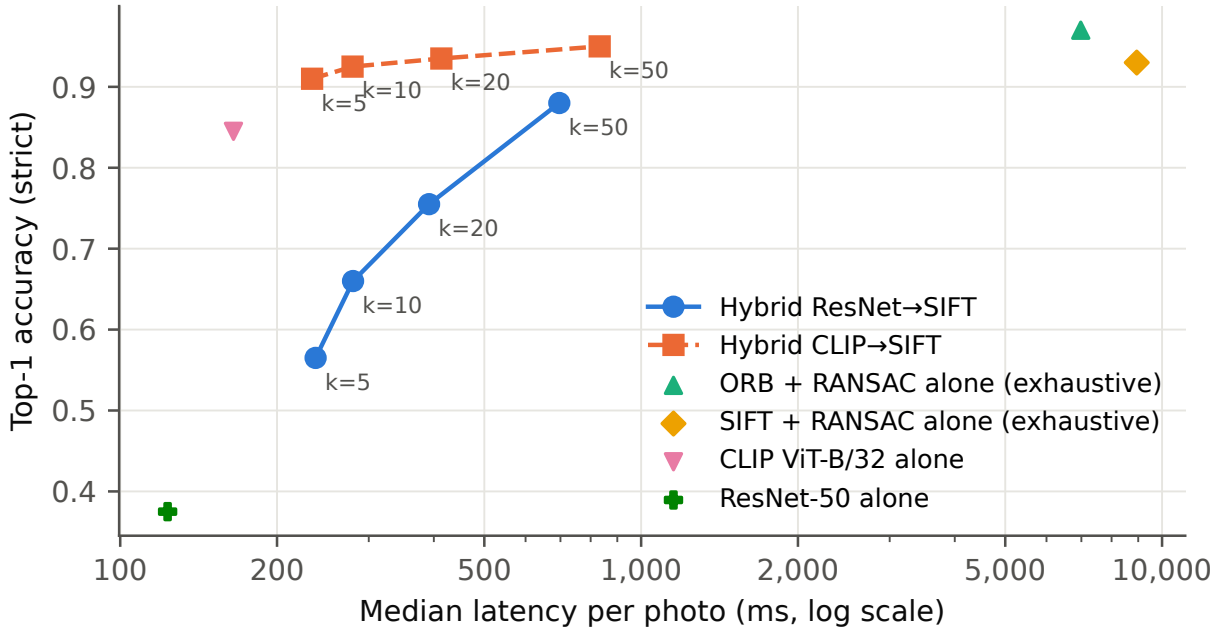


Figure 6.3. Hybrid shortlist size k (5, 10, 20, 50) at severity 3: accuracy versus median latency, compared with each component alone.

6.4 EFFECT OF RECTIFICATION

Table 6.2 compares each method at severity 3 with and without the rectification step. Without it, the global methods collapsed: the CLIP hybrid fell from 0.935 to 0.080, CLIP from 0.845 to 0.020, ResNet-50 from 0.375 to 0.005 and the pixel baseline from 0.900 to 0.005. In these photos the card covers only part of the image and is rotated and seen in perspective; resizing the whole photo leaves most of the image as background, which dominates a global descriptor. The keypoint methods degraded less: SIFT kept 0.425 (from 0.930) and ORB 0.165 (from 0.970). Their features are local and invariant to rotation and scale, and the RANSAC homography can itself absorb the perspective, so they do not need the card to fill the image; they still lose most of their accuracy, because the small, unrectified card provides far fewer keypoints. Rectification is therefore a necessary step for every method, and essential for the global ones.

Table 6.2. Ablation at severity 3: strict top-1 with rectification versus the whole photo resized to the canonical size.

Method	Rectified	Whole photo
Pixel thumbnail	0.900	0.005
ORB + RANSAC	0.970	0.165
SIFT + RANSAC	0.930	0.425
ResNet-50	0.375	0.005
CLIP ViT-B/32	0.845	0.020
Hybrid ResNet→SIFT	0.755	0.090
Hybrid CLIP→SIFT	0.935	0.080
ResNet-50 +rot	0.340	0.005
CLIP +rot	0.810	0.025
Hybrid ResNet→SIFT +rot	0.715	0.100
Hybrid CLIP→SIFT +rot	0.925	0.075

6.5 MULTI-CARD SCENES

Table 6.3 reports the scenes with 2–6 cards at severity 2. Detection is the same for every method. With no overlap the detector found 0.706 of the cards with a precision of 0.911; with up to 15% and 30% overlap, recall fell to 0.330 and 0.291. The best end-to-end result (card detected *and* correctly identified) was obtained by the CLIP hybrid: 0.643 without overlap, 0.285 and 0.247 with overlap. Among the cards that were detected, the CLIP hybrid identified 0.911 correctly without overlap, so the loss comes mostly from detection: overlapping cards merge into one contour, as expected from the design of the detector (Section 3.2).

Table 6.3. Multi-card synthetic scenes (severity 2): detection precision / recall / F1 (IoU $\geq$ 0.5) and end-to-end top-1 (detected and correctly identified / all cards).

Method	Overlap	P	R	F1	End-to-end top-1
ResNet-50	0%	0.911	0.706	0.795	0.349 [0.30, 0.40]
ResNet-50	15%	0.759	0.330	0.460	0.168 [0.13, 0.20]
ResNet-50	30%	0.721	0.291	0.415	0.154 [0.12, 0.19]
CLIP ViT-B/32	0%	0.911	0.706	0.795	0.571 [0.52, 0.62]
CLIP ViT-B/32	15%	0.759	0.330	0.460	0.267 [0.22, 0.31]
CLIP ViT-B/32	30%	0.721	0.291	0.415	0.220 [0.18, 0.26]
Hybrid ResNet→SIFT	0%	0.911	0.706	0.795	0.574 [0.53, 0.63]
Hybrid ResNet→SIFT	15%	0.759	0.330	0.460	0.275 [0.23, 0.32]
Hybrid ResNet→SIFT	30%	0.721	0.291	0.415	0.240 [0.20, 0.28]
Hybrid CLIP→SIFT	0%	0.911	0.706	0.795	0.643 [0.59, 0.69]
Hybrid CLIP→SIFT	15%	0.759	0.330	0.460	0.285 [0.24, 0.33]
Hybrid CLIP→SIFT	30%	0.721	0.291	0.415	0.247 [0.21, 0.29]

6.6 IDENTIFICATION ACROSS PRINTED LANGUAGES

In this experiment SIFT, CLIP and the CLIP hybrid were run on the same English, Spanish and Japanese queries (Table 6.4). All queries are at severity 2, so the in-plane rotation is at most $15°$ and the upside-down problem does not occur.

Table 6.4. Lenient top-1 identification per printed language (test split, severity 2) with 95% bootstrap intervals, at two photo resolutions: card about 530 px tall ("low") and about 1050 px tall ("phone", like a 12 MP photo). @art = only the artwork window is compared; +rot = the query is also tried rotated 180°. Queries: en 60, es 60, ja 43.

Method	Res.	English	Spanish	Japanese
SIFT + RANSAC	low	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	phone	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.977 [0.93, 1.00]
SIFT + RANSAC @art	low	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
	phone	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]
CLIP ViT-B/32	low	0.983 [0.95, 1.00]	0.883 [0.80, 0.95]	0.628 [0.49, 0.77]
	phone	1.000 [1.00, 1.00]	0.967 [0.92, 1.00]	0.767 [0.65, 0.88]
CLIP @art	low	0.933 [0.87, 0.98]	1.000 [1.00, 1.00]	0.767 [0.63, 0.88]
	phone	0.967 [0.92, 1.00]	1.000 [1.00, 1.00]	0.860 [0.74, 0.95]
Hybrid CLIP→SIFT +rot	low	0.983 [0.95, 1.00]	0.950 [0.88, 1.00]	0.930 [0.84, 1.00]
	phone	1.000 [1.00, 1.00]	0.950 [0.88, 1.00]	1.000 [1.00, 1.00]
Hybrid CLIP→SIFT @art +rot	low	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.884 [0.77, 0.98]
	phone	1.000 [1.00, 1.00]	1.000 [1.00, 1.00]	0.977 [0.93, 1.00]

- **SIFT + RANSAC** was almost unaffected by the printed language: its lenient top-1 was between 0.977 and 1.000 in every language and at both resolutions, with or without the artwork crop.
- **CLIP** depended on the language: at low resolution its lenient top-1 was 0.983 for English, 0.883 for Spanish and 0.628 for Japanese cards (phone resolution: 1.000, 0.967, 0.767). Restricting CLIP to the artwork window gave 0.767 (low) and 0.860 (phone) on Japanese cards, but with only 43 Japanese queries the confidence intervals of CLIP and CLIP@art overlap.
- **The CLIP hybrid** with SIFT re-ranking reached 0.930 (low) and 1.000 (phone) on Japanese cards; the artwork-only hybrid reached 0.884 and 0.977.
- **Cost.** The median time per photo at phone resolution, pooled over the three languages, was 109 ms for CLIP, 553 ms for the CLIP hybrid, 7,281 ms for SIFT@art and 33,688 ms for exhaustive SIFT (indicative only, see the latency note in Chapter 4). The index of SIFT needs 326.7 MB, the index of CLIP 1.6 MB.

6.7 PRINTED-LANGUAGE CLASSIFICATION

Table 6.5 and Figure 6.4 report the OCR-based language classifier. The first version of the rules, fixed before looking at any data and evaluated once on the test split at low resolution, reached an accuracy of 0.539: 66 cards were "unknown" and 17 were assigned to another language. The second version (kana-only script test, language-code box, phone resolution, OCR at 750×1050) was tuned on the development split (0.928) and then evaluated once on the test split: 0.911 [0.872, 0.950], with 14 "unknown" answers and 2 confusions between languages.

Table 6.5. Printed-language classification (English / Spanish / Japanese) with OCR + rules. Version 1 rules were fixed before seeing any data; version 2 was tuned on the dev split only. "Unknown" = no usable text; "wrong" = another language predicted.

Rules	Split	Res.	n	Accuracy [95% CI]	Unknown	Wrong
v1	test	low	180	0.539 [0.46, 0.61]	66	17
v2	dev	phone	180	0.928 [0.89, 0.96]	13	0
v2	test	phone	180	0.911 [0.87, 0.95]	14	2



Figure 6.4. Confusion matrices of the printed-language classifier on the test split (rows: true language; columns: predicted language or "unknown").

6.8 CONDITION ESTIMATE

Table 6.6 summarises the validation on synthetically damaged cards and Figure 6.5 shows an example. Centering was measured with a median error of 0.58 (left/right) and 0.70 (top/bottom)

percentage points. Corner cuts, corner whitening and edge whitening correlated clearly with their sub-scores (ρ = -0.74, +0.62, +0.78), edge fraying less so (ρ = +0.41). Surface defects were found with a recall of 0.61 and a precision of 0.69.

Table 6.6. Validation of the visual condition estimate on 80 test cards with synthetic damage of known strength (front side only). ρ = Spearman rank correlation between true damage strength and the estimate.

Measure	Result
Cards rectified / reference aligned	80 / 76
Centering error L/R, median / 90th pct. (percentage points)	0.58 / 2.33
Centering error T/B, median / 90th pct. (percentage points)	0.70 / 2.33
ρ corner cut vs. fill sub-score (expect < 0)	-0.74
ρ corner whitening vs. whitening sub-score	+0.62
ρ edge whitening vs. whitening sub-score	+0.78
ρ edge fray vs. fray sub-score	+0.41
Surface defects: recall / precision	68/111 = 0.61 / 72/105 = 0.69
Surface defects with correct type	59/111
ρ part score vs. true damage: corners / edges / surface / centering	-0.53 / -0.32 / -0.86 / -0.90
ρ overall score vs. total damage	-0.46

The back side could not be validated in the same way, because no official scans of card backs exist; the application's results on real photos follow in Section 6.9.

6.9 THE APPLICATION ON REAL PHOTOS

At the end of the project the CardDex collection contains 7 cards that were identified from real photos and saved by the author: 5 English cards and 2 Japanese cards from *Battle Partners* (Table 6.7; snapshot of 2026-11-08). Their stored prices add up to USD 42.28 for the English cards and EUR 2.93 for the Japanese cards (prices change daily; the two currencies are kept apart). Each saved card keeps the photo it was identified from (Figure 6.6). Most of them are phone photos of cards inside protective sleeves or rigid holders, lying on a keyboard, which is a realistic and difficult setting for the detector. The Mega Chandelure is a Japanese print of a *Pitch Black* card; Japanese scans of that set were not available, so it can only match its English version, and its image looks like a clean scan rather than a phone photo.

Re-identification. To turn the saved photos into a small real-photo test, every photo was identified again with each method, through the same code path as the application and against all 917 reference cards of the application (`report/app_evidence.py`). The ground truth is the card that the user saved, which the application lets the user correct before saving. Table 6.8 shows the result.



Figure 6.5. Condition estimate on a synthetically damaged test card. Left: the rectified photo. Right: the same photo with the analysed corner and edge zones and the detected surface defects marked.

Table 6.7. Cards saved in the CardDex collection (snapshot of 2026-11-08). Price when saved: English cards = mean TCGplayer market price of the card's variants in USD; Japanese cards = Cardmarket average in EUR (both via TCGdex). Score of the saved card with the method used at save time: RANSAC inliers for SIFT, ORB and the hybrids, cosine similarity in % for ResNet and CLIP. Condition: overall visual estimate (0–1000; mean of the two sides) and the front / back side scores as stored by the app (– = not assessed). Lang.: printed language stored with the item.

Card (id)	Rarity	Lang.	Method	Price	Score	Condition (overall; front / back)
Mega Chandelure ex (`me5-115`)	Special Illustration Rare	en	sift	USD 32.39	59	320; 167 / 473
Morpeko ex (`me5-102`)	Ultra Rare	en	hybrid CLIP +rot	USD 3.52	285	258; 269 / 246
Goldeen (`me5-87`)	Illustration Rare	en	sift	USD 3.51	202	150; 198 / 103
Toucannon (`me5-94`)	Illustration Rare	en	resnet	USD 1.55	82.9	132; 162 / 103
Thievul (`me5-92`)	Illustration Rare	en	resnet	USD 1.31	76.9	136; 170 / 103
Maractus (JP) (`ja-SV9-101`)	–	ja	sift	EUR 1.33	251	261; 220 / 302
Swinub (JP) (`ja-SV9-106`)	–	ja	sift	EUR 1.60	290	189; 197 / 181

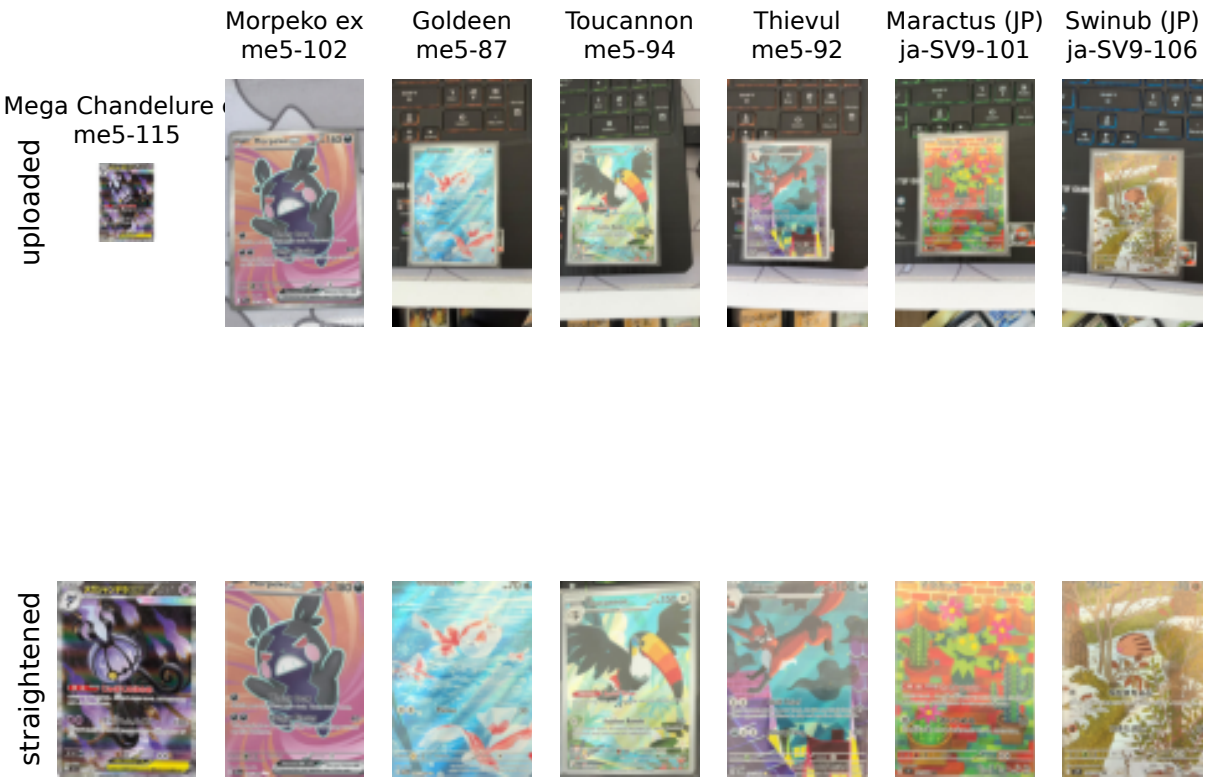


Figure 6.6. The photos stored with the saved cards (top) and the card image that the application straightened from each one and passed to the retrieval method (bottom).

- SIFT, ORB, CLIP and the CLIP hybrid ranked the exact saved card first for all 7 photos, including both Japanese cards, which were matched to their Japanese scan and not to their English twin.
- ResNet-50 was correct for 5 of 7 photos (6 under the lenient metric): it ranked the Morpeko ex at position 191, and it returned the English twin (`sv9-160`) for the Japanese Maractus.
- The artwork-only hybrid found the right artwork every time (7 of 7 lenient) but returned the English twin for both Japanese cards (5 of 7 strict). This is exactly what the method is designed to do: it ignores the text, and the text is the only difference between the two prints.
- SIFT separated the correct card clearly in every case, for example 202 RANSAC inliers for Goldeen against 65 for the second card, and 251 against 74 for the Japanese Maractus.
- Median time per photo: 22,329 ms for SIFT, 7,064 ms for ORB, 270 ms for CLIP and 1,015 ms for the CLIP hybrid. These timings were taken while the main experiments were running on the same laptop and are only indicative.

With 7 photos these numbers illustrate the behaviour on real photos but are not a statistical result.

Table 6.8. Re-identification of the photos saved in the application with every method (same path as the app: rectification, then ranking of all 917 reference cards of the app, English and Japanese). Each cell is the rank of the saved card (1 = correct top-1); † = wrong exact card but same artwork (lenient top-1, e.g. the English twin of a Japanese card); bold = wrong even under the lenient metric. Ground truth = the card the user saved. Top-1 = strict / lenient. Latency: median over the photos (measured while other experiments were running, indicative only).

Method	*Mega Chandelure ex*	*Morpeko ex*	*Goldeen*	*Toucannon*	*Thievul*	*Maractus (JP)*	*Swinub (JP)*	Top-1	ms
Pixel thumbnail	1	1	**2**	1	**22**	**32**	**7**	3/3	142
SIFT + RANSAC	1	1	1	1	1	1	1	7/7	22,329
ORB + RANSAC	1	1	1	1	1	1	1	7/7	7,064
ResNet-50	1	**191**	1	1	1	2†	1	5/6	275
CLIP ViT-B/32	1	1	1	1	1	1	1	7/7	270
Hybrid CLIP→SIFT +rot	1	1	1	1	1	1	1	7/7	1,015
Hybrid CLIP→SIFT @art +rot	1	1	1	1	1	2†	2†	5/7	735

Detection on real photos. The detector returned an outline for every saved photo, but on sleeved cards the outline is not always the card edge (Figure 6.6): for several cards the straightened

image is cropped slightly inside the card border, and for the Toucannon it includes part of the sleeve. A likely reason is that the detector keeps the strongest card-shaped rectangle, and a sleeve or rigid holder adds further rectangles just inside and outside the card edge. Identification was not affected, because the artwork and most of the card remained inside the warped image.

Condition estimates on real photos. The stored condition estimates were low for every card (overall between 132 and 320 out of 1000), and several fronts were reported with many surface "defects" (for example 113 for the Mega Chandelure, compared with the English scan, and 115 for the Japanese Maractus, compared with its own Japanese scan). The Goldeen, Toucannon and Thievul have identical back scores (103), which suggests that the same back photo was used for the three cards. The true condition of these cards was not recorded, so the estimates cannot be scored; they are discussed in Section 7.4.

6.10 REAL PHOTOS (SYSTEMATIC EVALUATION)

The planned systematic real-photo set (70–100 labelled phone photos per set under controlled variations, Section 4.7) was not collected within the project. The real-photo evidence of this report is therefore limited to the 7 photos saved in the application (Section 6.9); this is listed as a limitation in Chapter 8.

7 ANALYSIS AND DISCUSSION

7.1 FAILURE ANALYSIS

Every strict top-1 failure of the single-card experiment was assigned exactly one cause by fixed rules (`cardrec/failure_analysis.py`), checked in this order: *localisation* (outline IoU < 0.5), *rectification* ($0.5 \leq$ IoU < 0.8), *upside down* (only for rotation-sensitive methods, when the true rotation lies outside the window that the warp returns upright), *same artwork* (the prediction is a reprint), *lookalike* (the prediction has the same name), *heavy distortion* (severity 4–5 and none of the above) and *other*. Because the first matching rule wins, the counts show the earliest stage at which a query went wrong, not every problem it had. Table 7.1 and Figure 7.1 show the result; Figure 7.2 shows one example per cause.

Table 7.1. Cause of every strict top-1 failure on the synthetic single-card queries (all finished severities pooled; recomputed from the current logs). Each failure gets the first cause that applies, in the column order.

Method	*localization*	*rectification*	*upside down*	*same artwork*	*lookalike*	*heavy distortion*	*other*	*total*
Pixel thumbnail (S1–5)	164	10	43	16	0	53	21	307
ORB + RANSAC (S1–5)	161	7	0	14	0	68	0	250
SIFT + RANSAC (S1–5)	156	6	0	18	1	60	9	250
ResNet-50 (S1–5)	162	8	40	10	2	164	160	546
CLIP ViT-B/32 (S1–5)	163	7	40	21	7	136	18	392
Hybrid ResNet→SIFT (S1–5)	161	8	31	15	2	117	44	378
Hybrid CLIP→SIFT (S1–5)	163	6	38	12	1	84	7	311
ResNet-50 +rot (S1–5)	162	8	0	9	2	201	175	557
CLIP +rot (S1–5)	163	7	0	21	5	171	28	395
Hybrid ResNet→SIFT +rot (S1–5)	161	8	0	12	3	148	53	385
Hybrid CLIP→SIFT +rot (S1–5)	163	6	0	12	1	111	9	302

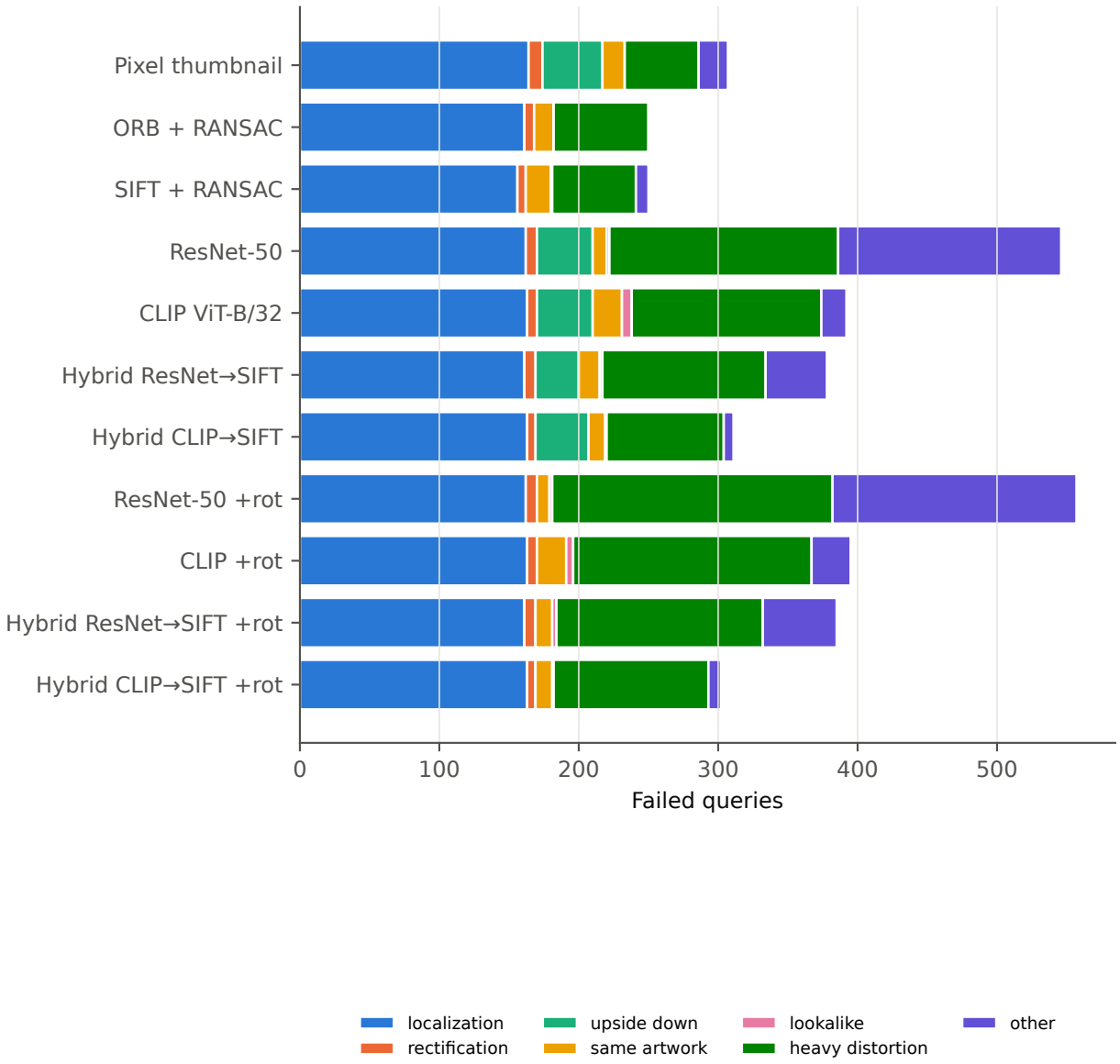


Figure 7.1. Failure causes of the strict top-1 errors per method (all severities pooled, 200 queries per severity).

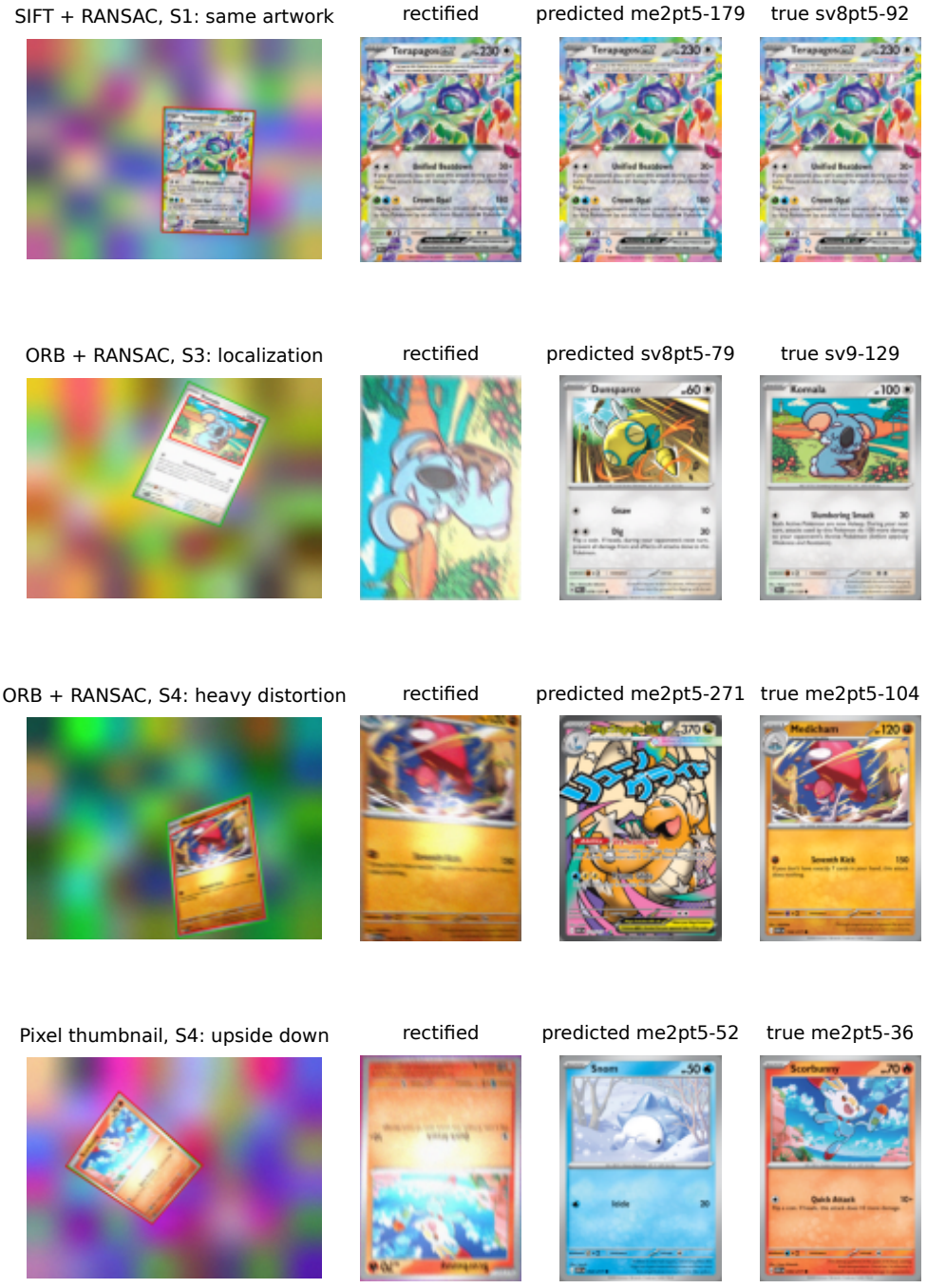


Figure 7.2. Failure examples, one per row: same artwork (SIFT + RANSAC, severity 1); localization (ORB + RANSAC, severity 3); heavy distortion (ORB + RANSAC, severity 4); upside down (Pixel thumbnail, severity 4). Columns: query (green = true outline, red = detected outline), rectified card, predicted reference, true reference.

Localisation is the main bottleneck under strong distortion. Every method shares the same detector and the same queries, so every method had about the same number of localisation failures (161 for ORB, 163 for CLIP, 163 for the CLIP hybrid). Of the 250 ORB failures, 161 were localisation failures, and 55 and 183 of all ORB failures happened at severities 4 and 5. When the outline was wrong, identification almost never succeeded: at severity 5, ORB identified 0.007 of the 146 queries with a wrong outline. When the outline was correct, ORB still reached 0.296 at severity 5 and 0.815 at severity 4. So there are two separate problems at high severity: the detector misses the card in most photos, and even a correctly rectified card is often too blurred and noisy for reliable keypoints (68 ORB failures are "heavy distortion" with a correct outline). Improving the detector (for example with a learned detector) would therefore help more than changing the retrieval method, but would not solve severity 5 alone.

Global embeddings also fail on well-rectified cards. Apart from reprints, the keypoint methods did not fail when the outline was correct and the distortion moderate (ORB: 0 failures of the category "other"). ResNet-50, in contrast, had 160 failures of the category "other" (correct outline, severity 1–3, not a reprint and not a lookalike) and 164 "heavy distortion" failures, against 18 and 136 for CLIP. A likely reason is that a frozen ImageNet descriptor summarises a card mostly by its overall colour and layout, and many cards of the same energy type and frame design look alike at that level. SIFT re-ranking repairs most of these errors: the CLIP hybrid had only 7 "other" failures and the ResNet hybrid 44.

Reprints and lookalikes are rare. At severities 1 and 2, all strict errors of ORB and SIFT were reprint confusions (Section 6.2). This is expected: a reprint has the same artwork, and the only visible differences are the set symbol, the set code and the collector number in the bottom strip, which are only a few pixels tall at 500×700. The lenient metric shows that the method found the right *artwork* in every one of these cases. A dedicated second stage that reads the collector number (OCR at higher resolution, as already done for the language) could separate them. Lookalike errors (same name, different artwork) did not occur at all for ORB and SIFT and were rare for the embeddings (7 for CLIP, 2 for ResNet-50, 1 for the CLIP hybrid), so the different versions of the same Pokémon are distinguished well.

Upside-down cards. Rotation-sensitive methods failed on cards that the warp returned upside down: 43 times for the pixel baseline, 40 for ResNet-50 and 40 for CLIP, all at the severities with large rotations, while ORB and SIFT had none because they are rotation invariant. The +rot variant removed these failures completely, but, as shown in Section 6.2, did not raise the overall accuracy, because other failures took their place.

7.2 HOW REALISTIC ARE THE SYNTHETIC RESULTS?

The pixel-thumbnail baseline, which compares 32×44 grey thumbnails, identified 0.970 of the queries at severity 1 and 0.900 at severity 3, more than ResNet-50 (0.375) and CLIP (0.845) alone at severity 3. Such a simple method can only work this well because each synthetic query is rendered

from the *same* scan that is in the database: once the card is correctly rectified, query and reference are almost the same image up to blur, noise and lighting. A real photo of a physical card differs from the scan also in printing, colour reproduction, surface reflections of holographic cards, sleeves and wear. The real photos saved in the application confirm this: there the pixel baseline identified only 3 of the 7 cards, while SIFT, ORB, CLIP and the CLIP hybrid identified all of them (Table 6.8). The synthetic benchmark is therefore an optimistic estimate that favours pixel-level similarity; it is reliable for comparing robustness to distortion and for the ranking of the methods that use local features or learned embeddings, but the absolute numbers, and especially the strength of the pixel baseline, do not transfer to real photos.

Two further points limit the comparison across severities: each severity uses a different random set of 200 cards, and the latencies were measured on a laptop that was running other jobs (including other experiments) at the same time, so they indicate orders of magnitude rather than exact times.

7.3 LANGUAGE, TEXT AND THE GLOBAL EMBEDDINGS

The cross-language experiment (Section 6.6) gives a first view of how the method families differ. SIFT + RANSAC was nearly language-independent, which suggests that the artwork keypoints dominate the RANSAC homography and the different text has little influence. CLIP lost accuracy on Spanish and especially Japanese cards, consistent with its embedding being influenced by the text regions; restricting it to the artwork window helped on Japanese cards but the difference is not statistically clear with 43 queries. The hybrid recovered most of the gap: the shortlist of CLIP only needs to *contain* the correct card, and SIFT then picks it. Two caveats apply: the Japanese ground truth was matched by artwork similarity, so Japanese cards whose artwork differs from the English print are under-represented; and the cards were shown at severity 2 only.

The same experiment shows the cost side of the trade-off. Exhaustive SIFT took a median of 33,688 ms per photo against 785 cards, which is too slow for interactive use and grows linearly with the database, whereas the CLIP hybrid needed about 553 ms. The k-sweep on the main benchmark confirms the trade-off (Figure 6.3): the CLIP hybrid reaches close to the accuracy of exhaustive keypoint matching at a fraction of its time, and its cost grows with k, not with the size of the database.

7.4 WHAT THE APPLICATION SHOWS ABOUT REAL PHOTOS

The 7 photos saved in the application are too few for statistical conclusions, but they are real photos of real cards, in sleeves, and they point in clear directions.

Identification transfers to real photos. SIFT, ORB, CLIP and the CLIP hybrid identified every saved photo exactly, including the two Japanese cards. ResNet-50 was again the weakest method: it failed badly on the full-art Morpeko ex, whose whole surface is illustration. With so few photos this is an indication, not proof.

Separate Japanese reference cards are needed for exact identification. Adding the Japanese *Battle Partners* cards to the application's database made it possible to identify a Japanese copy as the Japanese card (with its own number and price) instead of its English twin. Methods that see the whole card, text included, did this correctly; the artwork-only hybrid, which by design ignores the text, returned the English twin. The two ways of comparing cards therefore answer two different questions: "which artwork is this?" (artwork only, language independent) and "which exact printing is this?" (whole card, which needs reference scans in every language that should be recognised).

Sleeves move the detected outline, not the answer. On sleeved cards the detector often locked onto a rectangle slightly inside or outside the real card edge. Identification was robust to this, but the condition estimate is not, because it measures borders, corners and edges at the outline (see below).

Exhaustive SIFT is too slow for the application. With a median of 22,329 ms per photo, plain SIFT, which is still the application's default method, makes the user wait many seconds (the screenshot in Figure 3.2 shows a similar time), whereas the CLIP hybrid gave the same answers in about 1,015 ms. This supports making the CLIP hybrid the default method. To help the user choose, the application now shows a short description of every method next to the method selector.

The condition estimate does not transfer to real photos yet. On synthetic damage the estimate was validated (Section 6.8), but on the real photos it produced uniformly low scores and many surface "defects". Several causes are visible in the data. (i) Reference mismatch: the front of the Japanese Mega Chandelure was compared with the *English* scan, so the different text counts as damage. (ii) Real-photo conditions: the Japanese Maractus was compared with its own Japanese scan and still produced 115 defects, so reflections of sleeves and holders, the keyboard texture around the card and the inexact outline (above) also generate false defects and penalise edges and corners. (iii) The back side has no clean reference, so it is measured only from its own border colours, and the same back photo appears to have been reused for three cards. The estimate should therefore be presented as experimental. Fixing it requires photos of unsleeved cards on a plain background, as the application already asks for, and a comparison with the true condition of real cards.

Stored language. The two *Battle Partners* cards were stored with the language "ja", because the application now pre-selects Japanese when the matched reference card is Japanese. The Mega Chandelure, which was saved before that change, is still stored as "en" although it is a Japanese print; the application does not log whether a stored language came from the OCR classifier or from the user.

7.5 PRINTED LANGUAGE AND CONDITION

The language classifier improved from 0.539 to 0.911 on the test split. Version 1 failed in two ways: 66 photos gave no usable text at low resolution (“unknown”), and 16 English or Spanish cards were labelled Japanese because the OCR model invented kanji on blurry Latin text. Version 2 addressed both at once (higher resolution for OCR, and only kana count as Japanese evidence), so the separate contribution of each change was not measured. In version 2 almost all remaining errors are “unknown” answers rather than wrong languages (14 versus 2), which is the safer kind of error because the application lets the user correct it. The language-code rule decided only 1 of the 180 test photos, because the OCR model misreads the tiny code box.

For the condition estimate, centering and corner damage are measured reliably on synthetic damage, whereas edge fraying is weak ($\rho = +0.41$): fraying of less than about half a millimetre is only a few pixels wide even at 12 pixels per millimetre. The overall score correlates only moderately with the total damage ($\rho = -0.46$) because the weights that combine the parts are hand-set. The estimate is useful to flag visible problems, but it is not a substitute for a professional grade.

8 CONCLUSIONS, LIMITATIONS AND FUTURE WORK

8.1 CONCLUSIONS

This project built and evaluated a complete pipeline for identifying Pokémon trading cards in photos: a classical detector and rectifier, a reference database of 785 English cards with reprints grouped by artwork, seven interchangeable retrieval methods with two variants, a synthetic benchmark with exact ground truth, two extensions (printed language and a visual condition estimate), and a working application that also recognises the 132 Japanese cards of *Battle Partners*. The experiments support the following conclusions.

1. **Local features with geometric verification are the most accurate.** At every severity the most accurate method was exhaustive ORB or SIFT with RANSAC, and the two were statistically tied with each other (severity 3: 0.970 and 0.930; severity 4: 0.725 and 0.745), and at mild distortion their only errors were reprints with identical artwork, which the lenient metric shows to be correct at the artwork level. Their cost is several seconds per photo, growing with the size of the database.

2. **The CLIP hybrid is the best practical compromise.** Re-ranking a 20-card CLIP shortlist with SIFT raised CLIP from 0.845 to 0.935 at severity 3, matching exhaustive SIFT (0.930) in a few hundred milliseconds per photo instead of several seconds; with $k = 50$ it reached 0.950. A ResNet-50 shortlist is much weaker and needs a long list ($k = 50$: 0.880).

3. **Of the frozen global embeddings, CLIP is clearly better than ResNet-50** (0.845 versus 0.375 at severity 3), but neither should be used alone for exact identification: at severities 3 and 4 both were beaten even by a 32×44 pixel thumbnail on the synthetic queries (0.900 and 0.600).

4. **Rectification is essential, and detection is the bottleneck.** Without rectification the global methods failed almost completely at severity 3 and even the most robust method, SIFT, fell from 0.930 to 0.425; under strong distortion or with overlapping cards most errors come from not finding the card outline (outline correct for only 27.0% of the photos at severity 5; detection recall 0.291 with 30% overlap).

5. **Trying both orientations (+rot) does not pay off**: it removes the upside-down errors but not the total number of errors.

6. **The results transfer to real photos, except for the pixel baseline.** On the 7 real photos saved in the application, SIFT, ORB, CLIP and the CLIP hybrid identified every card, including two Japanese cards that were recognised as Japanese printings, while the pixel baseline identified only 3. The visual condition estimate, validated on synthetic damage, does not yet give credible scores on real photos of sleeved cards.

8.2 LIMITATIONS

- **Mostly synthetic evaluation.** The benchmark uses synthetic photos rendered from the reference scans themselves, which makes the task easier than real photos (Section 7.2). The planned systematic real-photo set was not collected; the real-photo evidence is limited to the 7 photos saved in the application.

- **Small and recent database.** 4 English sets with 785 cards (plus 132 Japanese cards in the application); a full collection has tens of thousands of cards, which affects both the accuracy of the global embeddings (more neighbours) and the cost of exhaustive matching.

- **Frozen embeddings.** ResNet-50 and CLIP were not adapted to cards, and CLIP's centre crop hides the top and bottom of the card.

- **Rectification ambiguity.** The rectified card can be upside down; the +rot variant removes these errors for the embedding methods but adds others.

- **Classical detector.** Overlapping cards merge into one contour, strong blur or glare removes the border edge, and sleeves move the detected outline.

- **Indicative latencies.** Timings were measured on a laptop that was running other jobs at the same time (Appendix A).

- **Condition estimate.** Validated only on synthetic damage of the front side; on real photos it gave implausibly low scores, the weights are hand-set, and the score is not an official grade.

8.3 FUTURE WORK

- Collect and label the systematic real-photo set (varied backgrounds, lighting, glare, sleeves, several cards per photo) to confirm the ranking of the methods on real data.

- Separate reprints with a second stage that reads the set code and collector number at high resolution.

- Replace or complement the classical detector with a learned detector or segmentation model, motivated by the localisation failures, overlapping cards and sleeves.
- Fine-tune the global embedding on card images (for example with synthetic augmentations) so that the shortlist of the hybrid can be shorter and faster.
- Make the condition estimate robust to sleeves and reflections, validate it against the true condition of real cards, and add a clean reference for the card back.
- Make the CLIP hybrid the default method of the application, add more Japanese and other non-English sets, and log the OCR language prediction separately from the language chosen by the user.

9 CONTRIBUTION STATEMENT

This is an individual project. Mariana Lozano is the only team member and carried out all parts of the project: the problem definition and experimental design, the reference database and the reprint grouping, the card detection and rectification, the implementation and integration of all retrieval methods, the synthetic data generator, the evaluation and failure analysis, the language and condition extensions, the web application, and this report.

External data, libraries and pretrained models are listed in the Acknowledgments; no external code was copied into the project beyond the use of these libraries through their public interfaces.

[TODO: Mariana: confirm or correct the two paragraphs above, and add the disclosure of AI assistants required by the course policy (which tools, and for which parts: code, experiments, writing).**]**

REFERENCES

Babenko, A., Slesarev, A., Chigorin, A., and Lempitsky, V. 2014. "Neural codes for image retrieval." *Proceedings of the European Conference on Computer Vision (ECCV)*, 584–599.

Baek, Y., Lee, B., Han, D., Yun, S., and Lee, H. 2019. "Character region awareness for text detection." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 9365–9374.

Canny, J. 1986. "A computational approach to edge detection." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8 (6): 679–698, 10.1109/TPAMI.1986.4767851.

Cao, B., Araujo, A., and Sim, J. 2020. "Unifying deep local and global features for image search." *Proceedings of the European Conference on Computer Vision (ECCV)*, 726–743.

Chandrasekhar, V. R., Chen, D. M., Tsai, S. S., Cheung, N.-M., Chen, H., Takacs, G., Reznik, Y., Vedantham, R., Grzeszczuk, R., Bach, J., and Girod, B. 2011. "The Stanford mobile visual search data set." *Proceedings of the ACM Conference on Multimedia Systems (MMSys)*, 117–122, 10.1145/1943552.1943568.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. 2009. "ImageNet: A large-scale hierarchical image database." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 248–255, 10.1109/CVPR.2009.5206848.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. 2021. "An image is worth 16x16 words: Transformers for image recognition at scale." *International Conference on Learning Representations (ICLR)*.

Douglas, D. H. and Peucker, T. K. 1973. "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature." *The Canadian Cartographer*, 10 (2): 112–122.

Efron, B. 1979. "Bootstrap methods: Another look at the jackknife." *The Annals of Statistics*, 7 (1): 1–26.

Fischler, M. A. and Bolles, R. C. 1981. "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography." *Communications of the ACM*, 24 (6): 381–395, 10.1145/358669.358692.

Gordo, A., Almazán, J., Revaud, J., and Larlus, D. 2016. "Deep image retrieval: Learning global representations for image search." *Proceedings of the European Conference on Computer Vision (ECCV)*, 241–257.

He, K., Zhang, X., Ren, S., and Sun, J. 2016. "Deep residual learning for image recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778, 10.1109/CVPR.2016.90.

Lowe, D. G. 2004. "Distinctive image features from scale-invariant keypoints." *International Journal of Computer Vision*, 60 (2): 91–110, 10.1023/B:VISI.0000029664.99615.94.

Nistér, D. and Stewénius, H. 2006. "Scalable recognition with a vocabulary tree." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Vol. 2, 2161–2168, 10.1109/CVPR.2006.264.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. 2019. "PyTorch: An imperative style, high-performance deep learning library." *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 8024–8035.

Philbin, J., Chum, O., Isard, M., Sivic, J., and Zisserman, A. 2007. "Object retrieval with large vocabularies and fast spatial matching." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 10.1109/CVPR.2007.383172.

Radenović, F., Iscen, A., Tolias, G., Avrithis, Y., and Chum, O. 2018. "Revisiting Oxford and Paris: Large-scale image retrieval benchmarking." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 5706–5715.

Radenović, F., Tolias, G., and Chum, O. 2019. "Fine-tuning CNN image retrieval with no human annotation." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41 (7): 1655–1668.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. 2021. "Learning transferable visual models from natural language supervision." *Proceedings of the 38th International Conference on Machine Learning (ICML)*, Vol. 139 of *Proceedings of Machine Learning Research*, 8748–8763.

Razavian, A. S., Azizpour, H., Sullivan, J., and Carlsson, S. 2014. "CNN features off-the-shelf: An astounding baseline for recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 806–813.

Rublee, E., Rabaud, V., Konolige, K., and Bradski, G. 2011. "ORB: An efficient alternative to SIFT or SURF." *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2564–2571, 10.1109/ICCV.2011.6126544.

Sivic, J. and Zisserman, A. 2003. "Video Google: A text retrieval approach to object matching in videos." *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 1470–1477, 10.1109/ICCV.2003.1238663.

Spearman, C. 1904. "The proof and measurement of association between two things." *The American Journal of Psychology*, 15 (1): 72–101.

Suzuki, S. and Abe, K. 1985. "Topological structural analysis of digitized binary images by border following." *Computer Vision, Graphics, and Image Processing*, 30 (1): 32–46.

Venkatesan, R., Koon, S.-M., Jakubowski, M. H., and Moulin, P. 2000. "Robust image hashing." *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, Vol. 3, 664–666.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. 2020. "Transformers: State-of-the-art natural language processing." *Proceedings of the Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 38–45.

APPENDIX A

REPRODUCIBILITY AND DETAILED RESULTS

A REPRODUCIBILITY AND DETAILED RESULTS

A.1 SOFTWARE AND HARDWARE

Python 3.12 on Windows 11; OpenCV, NumPy, pandas and matplotlib with the versions pinned in `requirements.txt`; PyTorch 2.11 with CUDA, torchvision 0.26, Transformers 5.20 and EasyOCR 1.7.2 for the deep-learning methods and OCR. The deep models ran on a laptop GPU (NVIDIA RTX 3050 Laptop, 4 GB); SIFT, ORB and the detector run on the CPU. Random seeds are fixed in `cardrec/config.py`, so the query sets and splits are reproducible.

A.2 COMMANDS

All commands are run from the project root with the project's virtual environment.

- Reference database and artwork groups: `python -m cardrec.database`, `python -m cardrec.art_groups`
- Synthetic experiments: `python -m cardrec.experiments --exp <name>`, with `<name>` one of `main`, `ksweep`, `norectify`, `scenes` or `all`
- Failure analysis: `python -m cardrec.failure_analysis`
- Language: `python -m cardrec.lang_data build|photos|eval [--split dev]`
- Condition validation: `python -m cardrec.condition_eval --n 80 --split test`
- Real photos: `python -m cardrec.real_photos labels|check|eval`
- Application: `python -m cardrec.app.server` (opens `http://127.0.0.1:5000`)
- Application evidence (saved collection, re-identification of its photos, screenshot crops): `python report/app_evidence.py`
- Tables, figures and numbers of this report: `python report/make_figures.py`

A.3 DETAILED RESULTS OF THE MAIN COMPARISON

Table A.1. All finished runs of the main experiment. Top-1 with 95% bootstrap interval (1000 resamples); MRR = mean reciprocal rank (= mAP, one relevant card per query); Rect. ok = fraction of queries whose detected outline has IoU $\geq$ 0.8 with the true outline; Top-1|ok = top-1 among those queries; latency = median per photo (shared PC, see text).

Method	Sev.	Top-1 [95% CI]	Top-5	MRR	Rect. ok	Top-1|ok	ms
Pixel thumbnail	1	0.970 [0.94, 0.99]	0.985	0.978	0.985	0.975	68
Pixel thumbnail	2	0.930 [0.90, 0.96]	0.960	0.945	0.990	0.939	74
Pixel thumbnail	3	0.900 [0.85, 0.94]	0.955	0.928	0.980	0.918	85
Pixel thumbnail	4	0.600 [0.53, 0.66]	0.690	0.639	0.890	0.674	95
Pixel thumbnail	5	0.065 [0.03, 0.10]	0.105	0.089	0.270	0.222	112
ORB + RANSAC	1	0.995 [0.98, 1.00]	1.000	0.998	0.985	0.995	6,109
ORB + RANSAC	2	0.975 [0.95, 0.99]	1.000	0.988	0.990	0.975	6,225
ORB + RANSAC	3	0.970 [0.94, 0.99]	0.995	0.983	0.980	0.974	6,981
ORB + RANSAC	4	0.725 [0.67, 0.79]	0.800	0.765	0.890	0.815	4,628
ORB + RANSAC	5	0.085 [0.05, 0.12]	0.130	0.113	0.270	0.296	3,340
SIFT + RANSAC	1	0.995 [0.98, 1.00]	1.000	0.998	0.985	0.995	33,042
SIFT + RANSAC	2	0.970 [0.94, 0.99]	1.000	0.985	0.990	0.970	13,298
SIFT + RANSAC	3	0.930 [0.89, 0.96]	0.970	0.952	0.980	0.934	8,941
SIFT + RANSAC	4	0.745 [0.69, 0.81]	0.785	0.764	0.890	0.820	3,141
SIFT + RANSAC	5	0.110 [0.07, 0.16]	0.165	0.136	0.270	0.333	1,291
ResNet-50	1	0.970 [0.94, 0.99]	0.995	0.983	0.985	0.975	105
ResNet-50	2	0.780 [0.72, 0.83]	0.930	0.849	0.990	0.788	108
ResNet-50	3	0.375 [0.31, 0.45]	0.585	0.476	0.980	0.372	123
ResNet-50	4	0.120 [0.07, 0.17]	0.260	0.197	0.890	0.129	195
ResNet-50	5	0.025 [0.01, 0.05]	0.050	0.048	0.270	0.056	223
CLIP ViT-B/32	1	0.970 [0.94, 0.99]	1.000	0.985	0.985	0.970	140
CLIP ViT-B/32	2	0.955 [0.93, 0.98]	0.995	0.975	0.990	0.960	141
CLIP ViT-B/32	3	0.845 [0.79, 0.90]	0.930	0.887	0.980	0.857	165
CLIP ViT-B/32	4	0.250 [0.18, 0.30]	0.445	0.349	0.890	0.275	154
CLIP ViT-B/32	5	0.020 [0.01, 0.04]	0.025	0.032	0.270	0.056	171
Hybrid ResNet→SIFT	1	0.995 [0.98, 1.00]	1.000	0.998	0.985	0.995	710
Hybrid ResNet→SIFT	2	0.940 [0.91, 0.97]	0.970	0.956	0.990	0.949	463
Hybrid ResNet→SIFT	3	0.755 [0.69, 0.81]	0.790	0.777	0.980	0.760	388
Hybrid ResNet→SIFT	4	0.370 [0.30, 0.44]	0.425	0.400	0.890	0.410	269
Hybrid ResNet→SIFT	5	0.050 [0.02, 0.09]	0.080	0.070	0.270	0.148	230
Hybrid CLIP→SIFT	1	0.995 [0.98, 1.00]	1.000	0.998	0.985	0.995	656
Hybrid CLIP→SIFT	2	0.965 [0.94, 0.99]	0.995	0.980	0.990	0.970	437
Hybrid CLIP→SIFT	3	0.935 [0.90, 0.96]	0.970	0.953	0.980	0.944	368
Hybrid CLIP→SIFT	4	0.530 [0.47, 0.60]	0.560	0.553	0.890	0.584	260
Hybrid CLIP→SIFT	5	0.020 [0.01, 0.04]	0.035	0.035	0.270	0.074	257
ResNet-50 +rot	1	0.970 [0.94, 0.99]	0.995	0.983	0.985	0.975	124
ResNet-50 +rot	2	0.745 [0.69, 0.80]	0.910	0.820	0.990	0.753	131
ResNet-50 +rot	3	0.340 [0.28, 0.41]	0.555	0.443	0.980	0.337	155
ResNet-50 +rot	4	0.130 [0.09, 0.18]	0.255	0.200	0.890	0.140	168
ResNet-50 +rot	5	0.030 [0.01, 0.06]	0.065	0.056	0.270	0.074	189
CLIP +rot	1	0.970 [0.94, 0.99]	1.000	0.985	0.985	0.970	135
CLIP +rot	2	0.950 [0.92, 0.98]	0.995	0.972	0.990	0.955	156
CLIP +rot	3	0.810 [0.76, 0.86]	0.920	0.862	0.980	0.821	138
CLIP +rot	4	0.275 [0.21, 0.34]	0.460	0.369	0.890	0.303	180
CLIP +rot	5	0.020 [0.01, 0.04]	0.035	0.036	0.270	0.056	201
Hybrid ResNet→SIFT +rot	1	0.995 [0.98, 1.00]	1.000	0.998	0.985	0.995	650
Hybrid ResNet→SIFT +rot	2	0.940 [0.91, 0.97]	0.965	0.954	0.990	0.949	489
Hybrid ResNet→SIFT +rot	3	0.715 [0.65, 0.78]	0.750	0.737	0.980	0.719	395
Hybrid ResNet→SIFT +rot	4	0.370 [0.30, 0.43]	0.400	0.394	0.890	0.410	286
Hybrid ResNet→SIFT +rot	5	0.055 [0.03, 0.09]	0.090	0.077	0.270	0.167	251
Hybrid CLIP→SIFT +rot	1	0.995 [0.98, 1.00]	1.000	0.998	0.985	0.995	678
Hybrid CLIP→SIFT +rot	2	0.965 [0.94, 0.99]	0.995	0.980	0.990	0.970	433
Hybrid CLIP→SIFT +rot	3	0.925 [0.89, 0.96]	0.960	0.943	0.980	0.934	365
Hybrid CLIP→SIFT +rot	4	0.565 [0.50, 0.63]	0.585	0.585	0.890	0.624	271
Hybrid CLIP→SIFT +rot	5	0.040 [0.01, 0.07]	0.055	0.056	0.270	0.148	245